A Secure and Auditable Framework for Legal Data Management: A Hybrid Approach Using Hyperledger Fabric and IPFS

Golam Bari Fardeen[a], Namare Shakib Angkon[a], Syed Zayan Anwar[a], Md. Masum Mizi[a], Shinthia Rahman[a], Md. Motaharul Islam[a,*]

[a] *United International University, United City, Madani Avenue, Dhaka, 1212, Bangladesh*

Abstract

Legal document management systems typically centralize trust in privileged administrators, making it difficult for external parties to verify whether access decisions or audit records have been altered. This paper presents a hybrid legal-data management framework that places Hyperledger Fabric on the active document-release path rather than using it only as a passive audit log: every protected download must pass a Fabric chaincode access check before ciphertext or wrapped document keys are released. If Fabric is unavailable, the download path fails closed and returns a denial instead of falling back to a database access-control list. The framework combines browser-side encryption, encrypted off-chain storage in a private IPFS swarm, and on-chain anchoring of document hashes and content identifiers. In experimental evaluation, the REST-gateway workload sustains up to 193 transactions per second, exceeding the estimated baseline for a 1,000-user firm; Fabric-based access checks add no statistically significant delay over a database-only check; and outage simulations confirm that successful downloads drop to zero during Fabric unavailability, with service resuming automatically on reconnection. The framework thus provides an empirical systems baseline for moving document-release decisions from application-only authorization to ledger-verifiable access-control state.

Keywords: Blockchain, Hyperledger Fabric, Legal document management, Data integrity, Access control, Digital forensics

*Corresponding author.

Email address: motaharul@cse.uiu.ac.bd (Md. Motaharul Islam)

1. Introduction

The legal industry depends on sensitive information. Law firms handle case strategy, client communications, privileged records, and electronic evidence, where confidentiality, integrity, and availability are essential. Yet many legal document management systems still rely on centralized infrastructure whose trust model has not kept pace with the threat landscape [1, 2].

Recent sector evidence motivates this security concern. A 2024 NetDocuments report found that more than half of identified data breaches at UK legal firms were caused by insiders [3]. The American Bar Association's 2023 Legal Technology Survey also found that 29% of respondents answered yes to a security-breach question [1]. These figures do not prove that audit logs were altered. They do, however, motivate the structural concern addressed here: NIST IR 8202 [4] identifies administrator control as a risk in permissioned systems, while NIST SP 800-92 [5] and NIST SP 800-53 AU-9 [6] emphasize protecting audit logs against unauthorized modification.

The core problem is architectural. Conventional legal document management systems centralize trust in a privileged database or application administrator. That administrator, or an attacker who compromises their credentials, may be able to read restricted documents, alter access-control tables, modify audit logs, or erase evidence of having done so [4]. A system may still encode a detailed role hierarchy, as illustrated in Fig. 1. The central question is not whether such a hierarchy can be represented in software, but whether its enforcement decisions and audit records can be verified by parties other than the administrator who recorded them.

1.1. Existing Challenges and Problem Definition

Three deficiencies in current legal document management systems motivate this work. First, audit trails are commonly stored in databases controlled by the same organization whose behavior they are supposed to evidence. Append-only database designs improve local tamper resistance. They do not provide independently verifiable immutability if verification still depends on the organization that controls the database [4].

Second, access control is usually enforced through folder permissions, application-layer role checks, or mutable ACL tables. A privileged administrator may bypass or alter these controls directly. There is often no verifiable

record of when access was granted, under what conditions it was meant to expire, or whether revocation was enforced [7, 8].

Third, many systems record a username alongside an event without a cryptographic signature binding the stated actor to the document content or action being recorded. This leaves room for fabricated or misattributed records [9].

These deficiencies share a root cause: the system's correctness depends on administrative trust. In adversarial multi-firm legal workflows, documents may be shared across firms, clients, experts, and regulators. Auditability therefore requires more than a database log controlled by one party. The required shift is from administrative trust to verifiable enforcement and evidence: access decisions should be reproducible from policy state, audit records should be independently checkable, and document integrity should be anchored cryptographically.

1.2. Proposed Solution and Research Objectives

This paper proposes a hybrid legal document management framework in which Hyperledger Fabric [10] is placed on the normal authorization path, rather than used only as a passive audit log. Under normal operation, a document download triggers a `CheckAccess` chaincode query. The query evaluates time-bounded grant state and capability-token semantics before the application releases ciphertext or a wrapped document key.

In this mode, the ledger does not merely record an application decision after the fact. It provides the verifiable policy state against which the access request is evaluated (cross-organization access; see Section 7 for intra-organization scope). When Fabric is unavailable, the evaluated document-download path returns HTTP 503 and does not fall back to a database ACL. Other Fabric-gateway paths follow the same fail-closed pattern by design but were not independently outage-tested. This prevents new document releases through the evaluated path during the tested Fabric outage.

Encrypted document payloads are stored off-chain in a private two-node InterPlanetary File System (IPFS) swarm [11]. SHA-256 hashes and content identifiers are anchored on-chain, keeping Fabric commit latency independent of document size. The ordering service is a 3-node EtcdRaft cluster distributed across FirmA, FirmB, and a regulator organization. This provides crash-fault tolerance under Raft assumptions, not Byzantine resistance to malicious orderer collusion [12, 13].

The framework design specifies browser-side AES-256-GCM encryption via the WebCrypto API [14, 15]. It also specifies an ECIES-style P-256 hybrid wrapping construction, implemented with WebCrypto-supported ECDH/KDF/AES-GCM primitives, for document-key wrapping. ECDSA P-256 document signatures are generated by the user [16]. The prototype implements these mechanisms partially and discloses the remaining engineering gaps in Table 7.

This work answers three research questions:

- **RQ1:** Can a permissioned blockchain architecture provide ledger-verifiable access-control state to consortium peers under normal operation, while enforcing a strict fail-closed policy for the evaluated document-download path during Fabric unavailability?
- **RQ2:** Does evaluating access control in chaincode at query time impose acceptable read-latency overhead, and does absolute write latency remain within the defined SLO for interactive legal workflows? We define acceptable overhead as median read overhead below 50 ms and total write latency below 5 s under realistic concurrency. The 50 ms threshold follows the cognitive heuristic that visual feedback under 100 ms is perceived as instantaneous [17, 18]. The 5.0 s write threshold is a conservative Service Level Objective (SLO) within the 10-second usability limit for background tasks before user context is lost [17].
- **RQ3:** Does the framework sustain throughput sufficient to absorb realistic legal workload spikes? The evaluation uses a synthetic steady-state baseline of a 1,000-user firm performing one document action per minute per user (16.7 TPS). This is an illustrative engineering assumption for capacity planning, not a measured industry workload [1, 19]. Because enterprise network traffic is bursty rather than uniform, a conservative 10× peak-load heuristic requires ≈167 TPS. Section 6.8 measures whether the tunable `BatchTimeout` configuration can reach this peak-load target.

Throughout the paper, we distinguish between the *proposed framework*, which describes the intended security architecture, and the *prototype*, which is the measured implementation. The prototype is useful for evaluating architectural viability and system costs, but it is not presented as a production-ready legal platform. Table 7 summarizes the gap between the complete framework design and the current implementation.

1.3. Contributions

The primary contributions of this work are fourfold. First, the core contribution is the measured relocation of the document-release authorization boundary from application-only trust to ledger-verifiable access-control state on the normal release path. The framework does not propose a new access-control primitive, a Byzantine-fault-tolerant ordering protocol, or a production-ready legal deployment. Fail-closed behavior is empirically confirmed for the download path.

Second, a proof-of-concept prototype is implemented and evaluated for a three-organization legal consortium using Hyperledger Fabric, a private two-node IPFS swarm, PostgreSQL operational storage, and a Spring Boot REST gateway. The evaluation uses the REST-gateway workload rather than an isolated peer/orderer microbenchmark, so the reported throughput reflects the application path exercised by legal document registration and retrieval.

Third, cryptographic mechanisms are integrated for document confidentiality, key distribution, integrity anchoring, and user-level signing. The framework uses AES-256-GCM for document encryption, ECIES-style P-256 for per-recipient key wrapping, and ECDSA P-256 for document signatures. Experiment 6 shows that the main measured advantage of ECIES in this setting is compact token size: 125 bytes per recipient compared with 256 bytes for RSA-OAEP 2048, a 51.2 % reduction. Timing results are reported as Node.js WebCrypto fallback measurements rather than browser-performance claims.

Fourth, an evidence-backed performance evaluation is provided. It covers throughput, latency, file-size effects, audit-query cost, synthetic WAN latency, `BatchTimeout` sensitivity, cryptographic operation cost, ledger-history depth, and outage behavior. At the canonical `BatchTimeout=2s` and `MaxMessageCount=500`, the REST-gateway workload sustains approximately 68-70 TPS, about 4.1 to 4.2× above the estimated 16.7 TPS legal workload. Reducing `BatchTimeout` to 500 ms produced the highest measured throughput, 193.0 TPS, for the evaluated three-organization deployment.

Fabric `CheckAccess` latency is statistically indistinguishable from the PostgreSQL-only ACL path after warm-up. Experiment 9 (Fig. 27) demonstrates strict fail-closed security enforcement on the document-download path: successful downloads fall to exactly zero for the full 45-second Fabric unavailability window ($t = 30$–75 s), with HTTP 503 denials sustained at 25–45 per second. Normal operation resumes automatically after an approximately 15-second gRPC reconnection lag.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 presents the system architecture. Section 4 describes governance, security, and the threat model. Section 5 details the prototype. Section 6 reports the experiments. Section 7 discusses limitations and regulatory considerations. Section 8 discusses the research-question findings and interprets the experimental results. Section 9 concludes the paper.

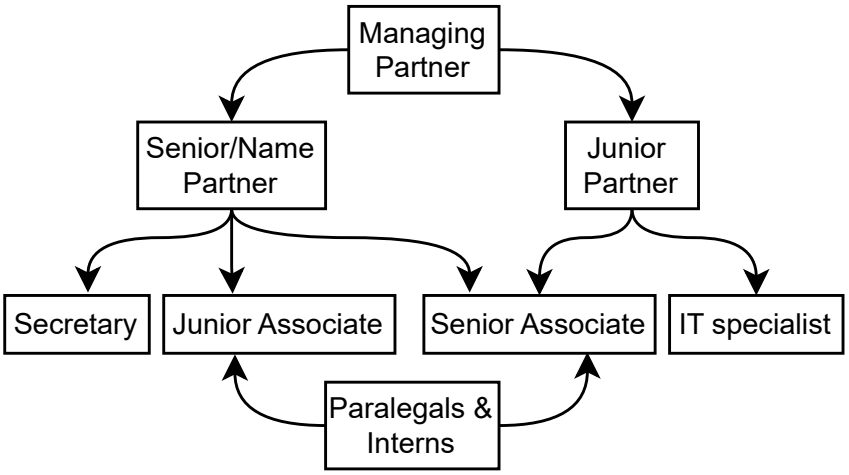


Figure 1: Role hierarchy enforced by the proposed framework, from managing partners with full case visibility to paralegals restricted to assigned documents only.

2. Literature Review

Blockchain applications in data management have expanded rapidly, evolving from integrity-preservation prototypes to systems that also address provenance, access control, and compliance workflows [20, 21]. Two recent systematic reviews frame the specific problem addressed in this paper. Alkhatib [22] reviewed blockchain-based identity and access management systems and found that independently verifiable audit trails and insider-aware access control remain open deployment challenges: many systems record access events after they occur, but still rely on a centralized application server or institutional database to decide whether an access request should be allowed. Precht et al. [23] similarly identify the absence of production-grade access-control enforcement in document management prototypes as a barrier to adoption. These reviews do not imply that prior systems lack all access control; rather,

they show that the enforcement boundary often remains outside the ledger, so an auditor must still trust the system that made the decision.

Table 1: Comparison with representative blockchain-based legal and evidence-management systems. ACL = access-control decision evaluated on the normal document-release path; Sig = user-level signature binding actor to document content; Rev = ledger-verifiable revocation or expiry. Y = supported; N = not supported; P = partial.

System	**Primary contribution**	**Tools**	**ACL**	**Sig**	**Rev**	**Main limitation relative to this work**
Liu and Zheng [24]	Judicial evidence preservation with on-chain anchoring and off-chain storage.	Blockchain + IPFS	N	N	N	Evidence integrity is anchored, but document access is not evaluated by ledger-visible policy before release.
Kim et al. [25]	Two-level evidence blockchain separating active and static investigation data.	Two-level blockchain	N	N	N	Improves evidence lifecycle organization, but does not provide query-time document-level chaincode authorization.
Onyeashie et al. [26]	Multi-party evidence workflow using Fabric channels and IPFS storage.	Fabric + IPFS	N	N	N	Supports workflow separation, but dynamic document ACL evaluation is not placed on every download path.
Verma et al. [27]	Tamper-evident judicial record management and timestamped action logging.	Public blockchain	N	N	N	Provides timestamping and integrity evidence, but does not target confidential document retrieval or private access enforcement.
Hanafi et al. [28]	Chain-of-custody auditing with Fabric and IPFS.	Fabric + IPFS	N	N	N	Uses Fabric mainly for audit recording; per-request access enforcement and per-recipient key wrapping are not evaluated.

Table 1 (continued)

System	Primary contribution	Tools	ACL	Sig	Rev	Main limitation relative to this work
Guo et al. [29]	Secure record storage and event auditing combining IPFS and Fabric.	Fabric + IPFS	N	N	N	Uses Fabric mainly for audit recording; per-request access enforcement and per-recipient key wrapping are not evaluated.
Capability and ABAC systems [30, 31, 32]	Fine-grained delegation or attribute-based authorization.	Capability / ABAC	P	N	Y	Policies can be expressive, but enforcement authority is typically a service layer rather than consortium-visible ledger state.
Proposed framework and prototype	Ledger-verifiable legal document authorization with encrypted off-chain storage.	Fabric + IPFS	Y	P	Y	Strict access uses Fabric fail-closed; prototype limits include custodial Fabric identities, custodial transaction attribution for user-level signatures, and 2-node IPFS swarm.

Table 1 compares prior systems based on where the access decision is made. The table does not imply that prior systems lack access control entirely. Instead, it distinguishes systems where access is mainly decided by the application server from systems where the decision is evaluated against ledger-visible policy before a document is released. The proposed contribution is therefore not a new access-control primitive, but a shift in the enforcement boundary: in normal operation, document access is checked through Fabric chaincode before protected material is returned to the user.

2.1. Foundational Technologies

Hyperledger Fabric is a permissioned blockchain platform in which participants are authenticated through Membership Service Providers (MSPs) that issue X.509 identities. Fabric supports configurable endorsement policies, private data collections, and channel-level governance, making it suitable for consortia of known institutions that require verifiable execution without

exposing all data to a public network [10, 33, 34, 35]. These properties are relevant to legal workflows because firms, clients, experts, and regulators often need shared evidence of actions without merging their internal databases.

Fabric is not designed to store large binary documents directly on the ledger. For that reason, many systems pair Fabric or another blockchain with IPFS [11]. IPFS provides content-addressed storage: if a stored object changes, its content identifier changes, making tampering detectable by address mismatch. However, content addressing does not by itself guarantee availability. Objects must be pinned and replicated by designated nodes, and private deployments require governance over who pins, who can fetch, and how long content is retained [36, 37]. The proposed framework therefore uses IPFS only for encrypted off-chain payload storage, while Fabric stores the verifiable metadata needed for integrity and access-control auditing.

Chaincode (Hyperledger Fabric uses the terms *chaincode* and *smart contract* interchangeably; this paper uses *chaincode* throughout) can automate policy decisions, but the security benefit depends on where the policy is evaluated. If chaincode only records an event after an application server has already made the access decision, the blockchain improves auditability but does not remove the application server from the authorization trust boundary. The distinction used in this paper is narrower: the proposed framework places a chaincode query on the normal document-download path so that the access decision is evaluated against ledger state before the application releases the protected material.

2.2. Evolution of Blockchain in Legal Technology

2.2.1. Integrity-Only Systems

Early legal and forensic blockchain systems focused primarily on tamper-evident chain of custody. NyaYa [27] anchors judicial records to establish timestamped evidence integrity, while Forensic-chain [38] and Block4Forensic [39] apply similar ideas to digital-forensics and IoT evidence pipelines. These systems demonstrate the value of immutable timestamps and hash anchoring, but they generally treat access control as an external concern. In other words, the ledger can help prove that a record existed at a given time, but it does not necessarily decide who may retrieve a confidential document in a multi-party legal workflow.

2.2.2. Hybrid Blockchain–IPFS Storage Systems

A second line of work combines blockchain anchoring with off-chain storage to address scalability. Liu and Zheng [24] and Jlil et al. [40] show that legal or criminal records can be represented by on-chain hashes while the larger payload is stored elsewhere. This pattern is useful because it avoids placing large documents directly on-chain. Its limitation is that storage integrity and access authorization are different problems. A document hash can prove that a retrieved payload matches a prior record, but it does not by itself enforce time-bounded sharing, revocation, or per-recipient key distribution.

2.2.3. Multi-Party and Fabric-Based Evidence Systems

More recent systems use permissioned blockchains to support multi-institution evidence workflows. Kim et al. [25] organize evidence across active and static chains, and Onyeashie et al. [26] use Fabric and IPFS to separate law-enforcement, forensic, and judicial workflows. Hanafi et al. [28] and Guo et al. [29] also combine IPFS and Fabric for secure record storage and event auditing, demonstrating the architecture's viability for highly confidential, role-based document sharing. These systems move closer to the legal-consortium model considered in this paper, but their primary contribution remains evidence management and audit recording. To the best of the authors' knowledge, none of the prior legal and evidence-management systems in Table 1 report evaluating an access-control decision against chaincode on the critical path of every document download or measuring the latency cost of doing so in a REST-gateway legal-document workflow, leaving an open baseline against which future permissioned-blockchain document-access systems can calibrate their access-control overhead.

2.3. Identified Research Gaps

The reviewed literature reveals four gaps that motivate the proposed framework. First, many blockchain-based evidence systems provide tamper-evident storage or timestamping but leave authorization to the application layer. This means a compromised or overly privileged application server may still authorize access using state that is not independently verifiable at the time of access. Second, access-control models are often coarse-grained or static. Legal collaboration requires operation-specific capabilities, expiry, revocation, and auditable sharing across institutions. Third, user-level non-repudiation is often incomplete: a ledger transaction may show that an organization submitted an event, but not that a specific user signed the document content

or action being recorded. Fourth, off-chain storage availability is frequently assumed rather than governed; IPFS content must be pinned and replicated deliberately.

The proposed framework addresses these gaps by moving the enforcement boundary onto the normal document-release path. The contribution is not a new access-control primitive, not a Byzantine-fault-tolerant ordering protocol, and not a production-ready legal deployment: it is a measured relocation of the document-release authorization boundary from application-only trust to ledger-verifiable access-control state on the normal release path, with fail-closed behavior empirically confirmed for the download path. The prototype still has limitations, including custodial Fabric identity handling and `localStorage`-based key storage, and those limitations are disclosed in Section 5 and Section 7.

Two related paradigms help clarify the boundary of the contribution. Capability systems such as Macaroons [30] and large-scale authorization systems such as Zanzibar [31] support expressive delegation and centralized policy evaluation at scale. The proposed framework does not replace those systems or claim to improve their policy expressiveness. Its difference is the audit and trust model: in the normal path, the policy state used for access evaluation is visible to consortium peers through Fabric rather than being only an internal service decision. Attribute-Based Access Control (ABAC) [41], including blockchain-assisted ABAC proposals such as Waheed et al. [42], can express fine-grained conditions. The present work is complementary: it focuses on where the decision is enforced and audited in a legal-document workflow, and on the measured overhead of placing that decision on the Fabric-backed access path.

Table 2 summarizes how the proposed architecture differs from these design classes across the properties most relevant to legal consortium deployments.

Table 2: Architectural Comparison of Access-Control Designs. This table compares the authorization trust boundary across four design classes. The centralized and append-only DB baselines are operational latency references, not security-equivalent alternatives.

Property	**centralized DB**	**Append-only DB**	**Fabric audit-only**	**This work**
Admin can silently alter access log	Yes	Hard	No (Fabric ledger; operational PostgreSQL log excluded)	No
Per-request ledger authorization on document-release path	No	No	No	Yes
Fail-closed on outage	No	No	No	Yes (empirically validated on document-download path; other paths fail-closed by design)
Ciphertext integrity verifiable by client	No	No	Partial	Yes ($\mathcal{H}_D$ anchored on-chain)
On-chain metadata leakage	None	None	Medium	Medium (encrypted content is protected, but ledger metadata and access patterns remain visible to channel members. PDC-based reduction is future hardening.)

Table 2 (continued)

Property	**centralized DB**	**Append-only DB**	**Fabric audit-only**	**This work**
Per-user transaction attribution	Yes	Yes	Custodial (prototype)	Custodial (prototype); per-user X.509 is production path
Byzantine-fault-tolerant ordering	N/A	N/A	No (CFT)	No (CFT); documented SmartBFT production path [43, 44]
Production-ready	Yes	Yes	Partial	No (proof-of-concept prototype)

3. System Architecture and Design

The proposed framework separates three cryptographically distinct concerns: document confidentiality (AES-256-GCM client-side encryption), access policy enforcement (chaincode-authoritative ACL), and audit integrity (dual-store: Fabric ledger and PostgreSQL append-only log). This section describes the operational workflows, the cryptographic layer, the two-layer access control architecture, and the audit architecture. Governance, consensus, and the formal threat model are in Section 4.

3.1. Operational Workflows

3.1.1. User Registration

Fig. 2 shows the registration workflow. The user submits identity attributes; the application server creates a profile in the off-chain PostgreSQL identity database and enrolls the user with the Fabric CA, issuing an X.509 certificate under the organization's MSP [45, 33]. In the framework design, each user also generates two keypairs in the browser at registration: an ECIES-style P-256 keypair (sk_{ecies}, pk_{ecies}) for document key wrapping and an ECDSA P-256 keypair (sk_{sign}, pk_{sign}) for document signing (Section 3.2). The two keypairs are intentionally distinct so that a key used for encryption is never used for signing, following key-separation practice [46]. In the measured prototype, Fabric transaction submission is performed through

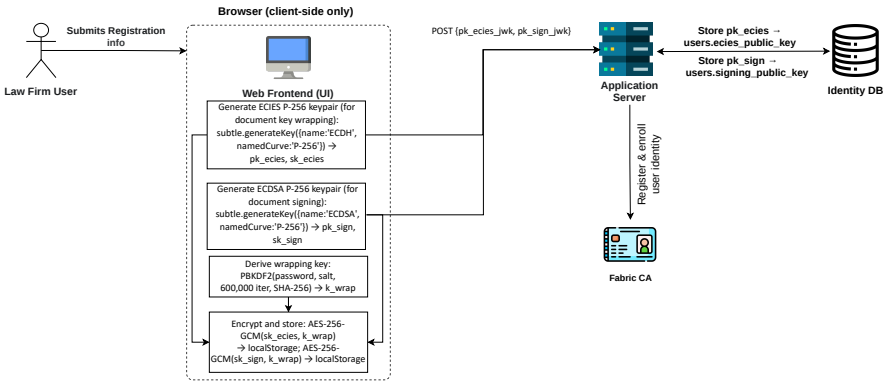


Figure 2: User registration workflow: browser-side P-256 keypair generation, PBKDF2-wrapped key storage in `localStorage`, public-key registration, and Fabric CA enrollment.

server-managed MSP credentials rather than per-user Fabric certificates; per-user X.509 enrollment is the production path.

3.1.2. Document Upload and Registration

Fig. 3 illustrates the upload workflow. The browser first generates a fresh AES-256-GCM document key k_{enc} and a 96-bit IV using `window.crypto.getRandomValues`. It computes the SHA-256 plaintext hash $\mathcal{H}_D$ *before* encryption so that $\mathcal{H}_D$ can serve as the document-integrity anchor.

After encryption, the browser prepends the IV to form the stored payload $D_{store} = IV \| D_{enc}$ and computes $\mathcal{H}_C = \text{SHA-256}(D_{store})$. This hash covers exactly the byte sequence from which the IPFS CID is derived. Strict client-side encryption ensures that the server receives only the IV-prepended ciphertext, not plaintext.

The signing key sk_{sign} then signs $\mathcal{H}_D \| \mathcal{H}_C \| \texttt{UTF-8}(id_D)$. This avoids a second round-trip for the CID while still binding the user to the exact plaintext content, the exact IPFS storage payload, and the document identifier. Because $\mathcal{H}_C$ hashes $IV \| D_{enc}$, any later attempt to swap the IPFS ciphertext invalidates the signature.

After IPFS returns the CID, the server invokes the `RegisterDocument` chaincode with $\mathcal{H}_D$, the CID, metadata, and σ. The prototype signature does not bind upload timestamp or owner metadata; that binding is a production hardening item [16]. Fabric transaction-level non-repudiation also remains custodial because the prototype submits transactions through server-managed MSP credentials (Section 5.1).

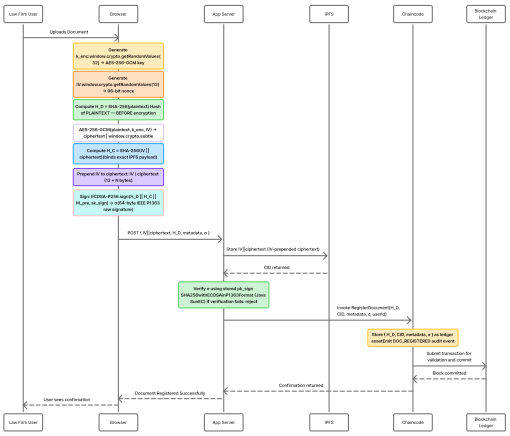


Figure 3: Document upload and on-chain registration, showing browser-side k_{enc} generation, plaintext hashing before encryption, sign-then-store ordering, IPFS upload, and CID anchoring.

3.1.3. Access Control and Document Retrieval

Access control is enforced in two layers. Fig. 4 shows Layer 1: Spring Security JWT RBAC validates the caller's role before the method body executes. Fig. 5 shows Layer 2: the `CheckAccess` chaincode evaluates the on-chain ACL at query time. Only if both layers authorize the request does the server retrieve the ciphertext from IPFS and the ECIES-wrapped key token from ledger storage. The client unwraps k_{enc} using sk_{ecies} and decrypts locally.

On the evaluated document-download path, Fabric unavailability causes HTTP 503 denial and no database ACL fallback. Other Fabric-gateway paths follow the same fail-closed pattern by design but were not independently outage-tested. The service records a `FABRIC_OUTAGE_ACCESS_DENIED` event in the operational PostgreSQL audit log. Because Fabric is unavailable at the time of denial, these outage events are not synchronously committed to the ledger and therefore inherit operational-log trust limits rather than ledger-verifiability guarantees. This prevents new document releases during the tested Fabric outage and prevents bypass of the chaincode authority [47, 10].

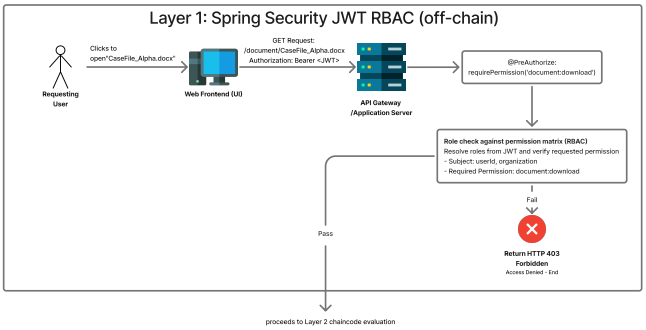


Figure 4: Layer 1 access control: Spring Security JWT RBAC at the API gateway. Requests failing role validation are rejected before reaching the blockchain layer.

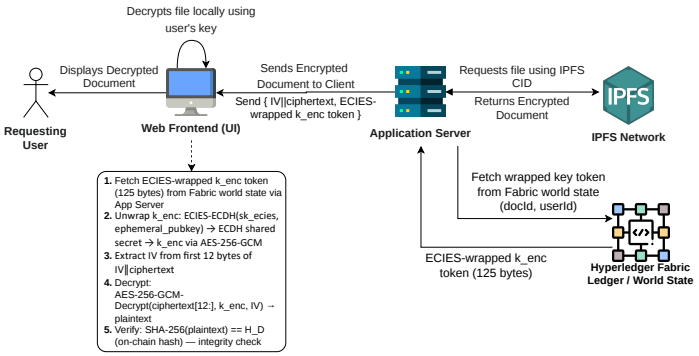


Figure 5: Layer 2 access control and document retrieval: `CheckAccess` chaincode evaluates the on-chain ACL at query time. The client decrypts locally; the server handles only ciphertext and ECIES-wrapped key tokens stored as encrypted ledger/world-state values.

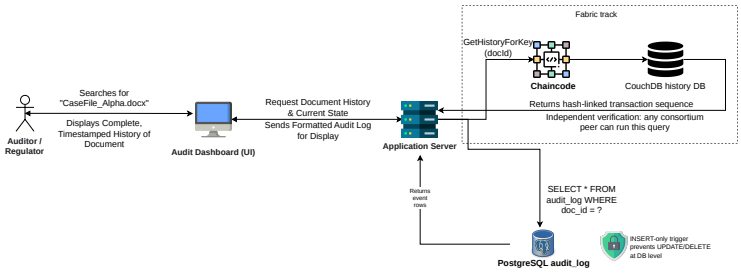


Figure 6: Audit trail retrieval. `GetHistoryForKey` returns the full transaction history for a document key and is independently verifiable by any consortium peer without trusting the application server.

3.1.4. Audit and Integrity Verification

The system records every security-relevant action to two independent stores: the Fabric ledger and the PostgreSQL `audit_log` table. Fabric is the independently verifiable record because any consortium peer can query and validate ledger history. PostgreSQL is the operational audit index, configured as INSERT-only by triggers and used for low-latency dashboard queries. The two stores provide different trust guarantees: PostgreSQL improves operational visibility, while Fabric provides the stronger evidence against a malicious or compromised database administrator. Figs. 6 and 7 show the audit query and hash verification workflows.

3.2. Cryptographic Layer

The framework uses a three-tier hybrid cryptographic design. Symmetric encryption (AES-256-GCM) protects document content. Asymmetric key wrapping using the Elliptic Curve Integrated Encryption Scheme (ECIES) over P-256 distributes document keys to authorized recipients. Digital signatures (ECDSA P-256) bind each document action to the acting user's identity. Fabric MSP infrastructure uses ECDSA P-256 for transaction signing independently [33, 16]. In the measured prototype, user-level document signatures and Fabric transaction identity should therefore be interpreted separately because Fabric transactions are submitted through server-managed MSP credentials.

3.2.1. Notation

Algorithm 1 formalizes the registration process using the sign-then-store pattern: in the prototype, σ is computed over $\mathcal{H}_D \| \mathcal{H}_C \| \texttt{UTF-8}(id_D)$ before

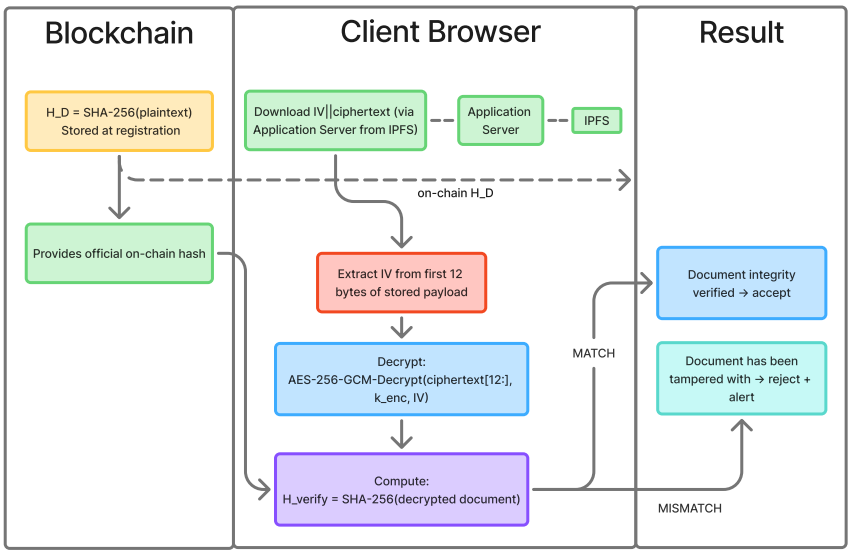


Figure 7: Document integrity verification by comparing on-chain $\mathcal{H}_D$ with the SHA-256 hash of the locally decrypted document.

the IPFS upload, avoiding the circular dependency of signing a value not yet known. The CID is submitted alongside σ in the Fabric transaction; any party verifying non-repudiation verifies σ over $\mathcal{H}_D \| \mathcal{H}_C \| \texttt{UTF-8}(id_D)$. With 96-bit IVs generated via a CSPRNG and a fresh AES-256-GCM key per document (Algorithm 1, lines 1–2), the birthday-bound IV collision probability for any single key is negligible below 2^{32} documents ($< 2^{-32}$ per NIST SP 800-38D [48]); per-document key generation eliminates cross-document IV-reuse risk entirely: because each document uses an independent k_{enc}, an IV collision in one document has no cryptographic effect on any other document [48]. Listing 2 (Appendix Appendix A) shows the browser implementation.

Protocol composition security. The ECIES-style key-wrapping construction is *intended* to achieve IND-CCA2 security under the Gap Diffie-Hellman (GDH) assumption following Shoup's formal model [49, 50]; however, the prototype composes ECDH, HKDF-SHA-256, and AES-256-GCM via WebCrypto without a named ECIES primitive, and the formal IND-CCA2

Table 3: Notation

Symbol	Description
D, M, U	Document, Metadata, User
sk, pk	Private and public key
k_{enc}	Per-document AES-256-GCM symmetric key
sk_{ecies}, pk_{ecies}	ECIES-style P-256 keypair for key wrapping
sk_{sign}, pk_{sign}	ECDSA P-256 keypair for signing
$\mathcal{H}(\cdot)$	SHA-256 hash function
$\mathcal{H}_D$	SHA-256 hash of plaintext document (plaintext hash; integrity anchor)
$\mathcal{H}_C$	SHA-256 hash of the IV-prepended ciphertext payload, $\mathcal{H}_C = \text{SHA-256}(IV \Vert D_{enc})$; covers the exact byte sequence stored in IPFS
σ	ECDSA P-256 digital signature
M_{pre}	Prototype signing payload component: $\texttt{UTF-8}(id_D)$ only. Framework design goal: deterministically serialized full metadata $\{id_D, \text{owner}, \text{filename}, t_{\text{now}}\}$; deferred to future work.
$\Vert$	Concatenation
$\perp$	Failure or null return

reduction for this specific composition is left as future work. Table 4 defines the 125-byte token format used in the prototype.

ECDSA P-256 achieves EUF-CMA under the elliptic curve discrete logarithm (ECDLP) assumption [16]. In this setting, an adversary cannot forge a valid signature without the signing key.

The registration workflow uses a sign-then-store ordering. Before the IPFS upload, σ is computed over three values: the 32-byte plaintext hash ($\mathcal{H}_D$), the 32-byte IV-prepended ciphertext-payload hash ($\mathcal{H}_C = \text{SHA-256}(IV \Vert D_{enc})$), and $\texttt{UTF-8}(id_D)$. This ordering avoids a circular dependency: the CID is not known until after upload, so signing a payload that includes the CID would require either a second round-trip or a separate commitment scheme.

Signing $\mathcal{H}_C$ still commits the user to the exact IPFS storage payload,

Table 4: ECIES-Style 125-Byte Wrapped-Key Token Format.

Field	Content	Bytes
Ephemeral public key	Uncompressed P-256 point	65
AES-GCM nonce	96-bit random IV	12
Wrapped key ciphertext	AES-256-GCM encrypted k_{enc}	32
Authentication tag	AES-256-GCM tag	16
Total		**125**

KDF: HKDF-SHA-256 over the ECDH shared secret. AAD: empty in the prototype; binding the recipient public key as AAD is a production hardening item.

including the prepended IV from which the CID is derived. An attacker therefore cannot swap the stored ciphertext later without invalidating the signature. The resulting construction binds both plaintext and ciphertext hashes, which is stronger than signing only plaintext while preserving the non-repudiation property of plaintext signing [16, 50].

The prototype relies on standard assumptions for ECIES-style wrapping, AES-GCM encryption, and ECDSA signatures, but it does not claim a new formal reduction for their exact WebCrypto composition. ECIES protects k_{enc} in transit [50]; AES-GCM protects document content at rest [14]; and ECDSA provides the user-key binding [16].

Algorithm 1 Document Registration (Sign-Then-Store Pattern)

Require: Document D, User U, Metadata M, keys sk_{sign}, pk_{ecies}
Ensure: Registration token R or $\perp$

$k_{enc} \leftarrow$ `getRandomValues`(32) {Fresh key per document}
$IV \leftarrow$ `getRandomValues`(12) {96-bit IV, unique per key}
$\mathcal{H}_D \leftarrow$ SHA-256(D) {Hash *plaintext before* encryption}
$D_{enc} \leftarrow$ AES-256-GCM(D, k_{enc}, IV)
$D_{store} \leftarrow IV \| D_{enc}$
$\mathcal{H}_C \leftarrow$ SHA-256(D_{store}) {Covers exact IPFS payload; IV included}
$M_{pre} \leftarrow$ `UTF-8`(id_D) {Prototype: document ID only. Framework goal: bind owner, filename, and t_{now} via deterministic serialization [51]}
$\sigma \leftarrow$ ECDSA-P256.Sign($\mathcal{H}_D \| \mathcal{H}_C \| M_{pre}$, sk_{sign}) {Sign plaintext AND ciphertext hashes} {Sign *before* IPFS upload}
$h_{IPFS} \leftarrow$ IPFS.Store($IV \| D_{enc}$) {Obtain CID after upload}
if $h_{IPFS} = \perp$ **then**
    return $\perp$ {Abort if IPFS unavailable}
end if
$M^* \leftarrow M \cup \{id_D, \mathcal{H}_D, \mathcal{H}_C, h_{IPFS}\}$ {Enrich with CID}
$\tau \leftarrow$ Blockchain.Submit($\langle M^*, \sigma, U \rangle$) {CID in tx; sig covers $\mathcal{H}_D \| \mathcal{H}_C \|$ `UTF-8`(id_D)}
if $\tau =$ success **then**
    emit `DOC_REGISTERED`($M^*.id$, U)
    return $R(M^*.id, h_{IPFS})$
else
    return $\perp$
end if

3.2.2. Key Derivation

User private keys are protected at rest using PBKDF2-SHA256 at 600,000 iterations, following current OWASP password-storage guidance [52] and exceeding the minimum work factor recommended by NIST SP 800-132 [53]. Listing 3 (Appendix Appendix A) shows the derivation function. Experiment 6 (Section 6.6) measures PBKDF2-SHA256 at 600,000 iterations with a P50 latency of 100.44 ms over 10 Node.js WebCrypto fallback trials. Browser automation was unavailable in the measurement environment, so this timing is reported as a same-API Node WebCrypto fallback result rather than a browser-performance claim.

3.2.3. ECDSA Document Signatures

At registration, the browser generates two separate P-256 keypairs using `window.crypto.subtle.generateKey`. The ECIES keypair is used for document key wrapping ($sk_{ecies} \neq sk_{sign}$), while the ECDSA keypair is used for signing. This follows key-separation practice [46].

When a user signs a document, the browser decrypts sk_{sign} in memory, imports it as a non-extractable signing key, and calls `window.crypto.subtle.sign( {name:'ECDSA',hash:{name:'SHA-256'}})`. In the prototype, the ECDSA-P-256 signature is computed over the fixed-width concatenation

$$P = \mathcal{H}_D \parallel \mathcal{H}_C \parallel \texttt{UTF-8}(id_D), \tag{1}$$

where $\mathcal{H}_D$ is the 32-byte SHA-256 plaintext hash, $\mathcal{H}_C$ is the 32-byte SHA-256 hash of the IV-prepended ciphertext, and $\texttt{UTF-8}(id_D)$ is the document identifier.

This payload binds the signing key to the exact document content and IPFS storage payload. Timestamp, owner identity, filename, and case metadata are not included in the prototype signed payload. Binding full document metadata through deterministic canonicalization, such as JCS [51], is a framework design goal and production hardening item.

The resulting signature is a 64-byte IEEE P1363 raw signature ($r\|s$, 32 bytes each for P-256). The server verifies it with `SHA256withECDSAinP1363Format` (Java SunEC, standard in Java 17+), which consumes WebCrypto's raw output without format conversion [16]. Before verification, the backend recomputes $\mathcal{H}_C = \text{SHA-256}(IV \| D_{enc})$ from the IPFS-retrieved bytes and reconstructs the payload in (1). Listing 4 (Appendix Appendix A) shows the server-side verifier.

3.2.4. Key Management Lifecycle

The key lifecycle spans five phases, shown in Figs. 8–12. Figures 8–11 depict mechanisms implemented in the prototype; Figure 12 (Phase 5, key-escrow recovery) depicts the framework design only and is not implemented in the current prototype (see Section 7.2 and Table 7).

Phase 1: key generation and storage. The browser derives a wrapping key from the user's password using PBKDF2. It uses that key to encrypt both sk_{ecies} and sk_{sign} before storing them in `localStorage`. The server never holds either private key in plaintext.

Phase 2: document encryption and registration. The browser generates k_{enc}, encrypts the document, and submits $\mathcal{H}_D$ and the CID to the `RegisterDocument` chaincode.

Phase 3: access grant. When an owner grants access, the browser wraps k_{enc} under the recipient's pk_{ecies} using an ECIES-style P-256 hybrid wrapping construction implemented with WebCrypto-supported ECDH/KDF/AES-GCM primitives. The resulting 125-byte wrapped token is submitted to `GrantAccess`; it is 51.2 % smaller than the 256-byte RSA-OAEP 2048 wrapped-key token measured in Experiment 6 [54].

Phase 4: access and decryption. The recipient unwraps k_{enc} using sk_{ecies}, extracts the IV from the first 12 bytes of the stored ciphertext, and decrypts locally.

Phase 5: revocation and rotation. On revocation, the system sets `key_rotation_pending` and notifies the document owner. The owner's browser must complete re-encryption because the server does not hold k_{enc}. An interim production-compatible path is proxy re-encryption, where the server holds a re-encryption key that transforms ciphertext for a revoked user into ciphertext for a newly authorized user without exposing plaintext [55, 56].

The `key_rotation_pending` window is a known limitation of client-managed key architectures. Once revocation is on the ledger, `CheckAccess` immediately denies new downloads for the revoked user. However, cached ciphertext remains outside cryptographic control until the owner completes re-encryption. Other authorized users can still be served the existing ciphertext under the original k_{enc} during this window. This limitation is disclosed in Table 7 and motivates the proxy re-encryption production path.

Two prototype limitations remain. First, Fabric MSP wallets are managed under a shared administrative identity rather than per-user X.509 certificates; the production path is per-user HSM-backed enrollment [45, 57]. Second, ECDSA signing keys are held in `localStorage` under PBKDF2 protection; the production path is WebAuthn/FIDO2 hardware-backed key storage [58, 59].

3.3. Two-Layer Access Control Architecture

Access control is enforced in two independent layers, both of which must grant a request before any ciphertext is served.

Layer 1 is enforced by Spring Security at the API gateway using JWT-validated role permissions [60, 47]. The `@PreAuthorize` annotation on the service method rejects requests that fail role validation before any business logic executes.

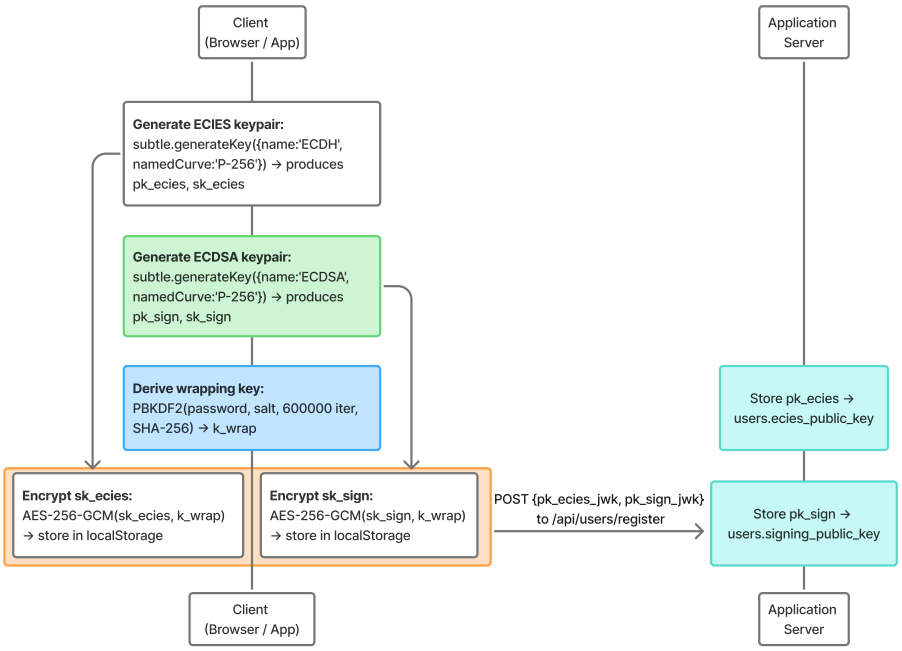


Figure 8: Phase 1: Client-side key generation. ECIES/ECDSA keys wrapped via PBKDF2-derived key and stored in `localStorage`, per NIST SP 800-131A key-separation practice.

Layer 2 is enforced by the `CheckAccess` chaincode invoked within the service method. The chaincode evaluates access through a single `ACL` map using two key conventions: per-user grants are stored under the user identifier (`ACL[userID]`), and organization-level grants are stored under a prefixed key (`ACL["ORG:"+userOrg]`). Both per-user and organization-level grants support time-bounded expiry evaluated at query time. The owner organization retains implicit read access via the `doc.OwnerOrg == userOrg` fallback. To prevent Multi-Version Concurrency Control (MVCC) conflicts and ledger bloat under high concurrent read loads, the chaincode dynamically evaluates expiry but does not attempt a state write-back during the read path. To maintain strict audit fidelity in the CouchDB world state without introducing read-path latency, the system design relies on a dedicated, asynchronous administrative chaincode invocation (`CleanExpiredTokens`) executed periodically as a back-

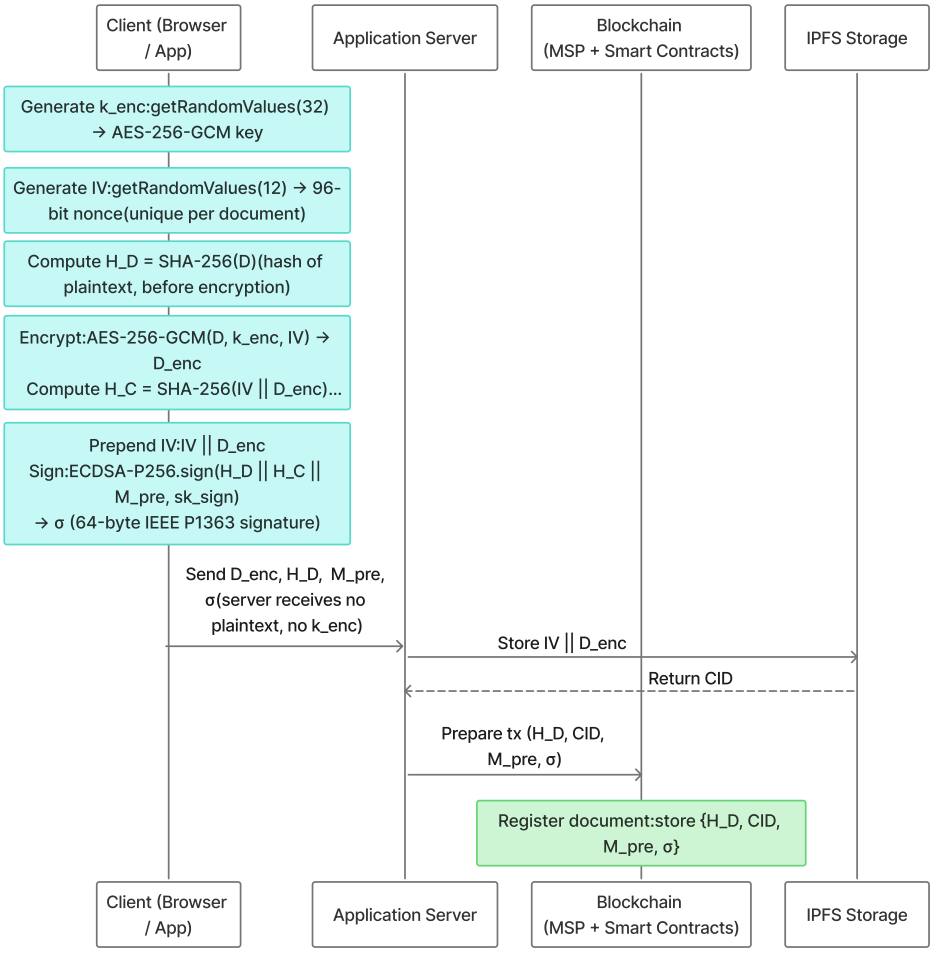


Figure 9: Phase 2 browser-side document encryption and registration with plaintext hashing before encryption, IV-prepended ciphertext, and ECDSA sign-then-store ordering.

ground cron-job to formally deprecate expired tokens. Listing 1 shows the chaincode implementation. Token deprecation therefore follows an *eventual consistency* model: expiry is enforced immediately at chaincode evaluation time, but the CouchDB world state reflects `StatusExpired` only after the next `CleanExpiredTokens` invocation; a direct world-state query between these two points will show the token as `StatusActive` despite being denied

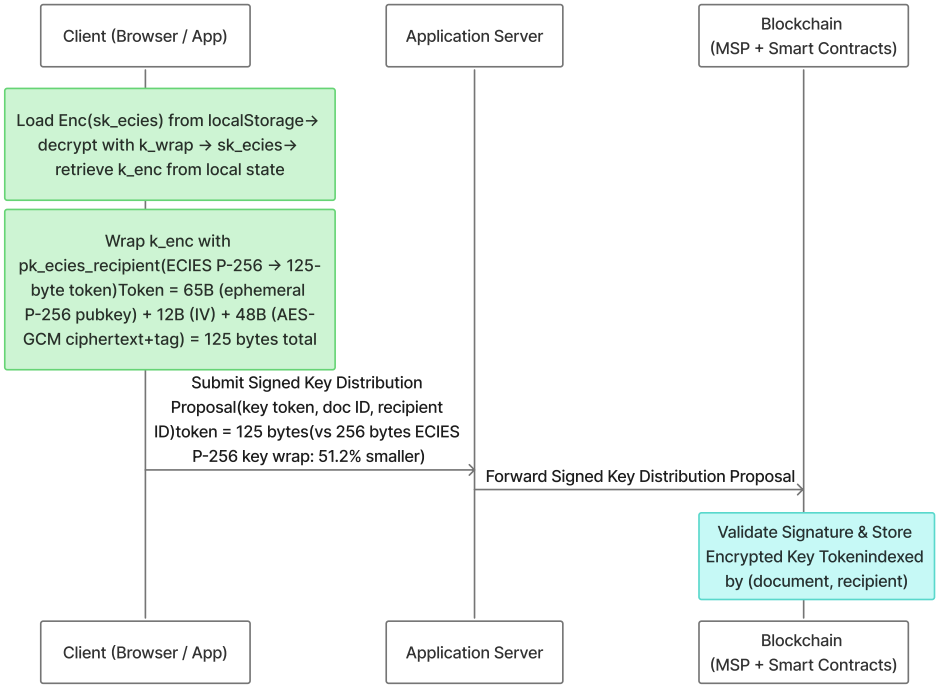


Figure 10: Phase 3 ECIES-style P-256 key wrapping, showing owner-wrapped k_{enc}, on-chain `GrantAccess` storage, and ECIES/RSA-OAEP token-size comparison.

at runtime.

Listing 1: `CheckAccess` chaincode enforcing time-bounded capability tokens (see Section 3.3 for the eventual-consistency model). All five access branches shown: per-user grants (Branches 1–2), owner-organization fallback (Branch 3), and org-prefix grants (Branches 4–5).

```
// chaincode/legalcc/chaincode.go
func (c *LegalContract) CheckAccess(
    ctx contractapi.TransactionContextInterface,
    docID, userID, userOrg string,
) (string, error) {
  doc, err := getDocument(ctx, docID)
  if err != nil { return "false", err }
  if doc.Status != StatusActive {
    return "false", nil
  }
  txTimestamp, err := ctx.GetStub().
      GetTxTimestamp()
```

```
  if err != nil {
    return "", fmt.Errorf(
        "failed to get tx timestamp: %w", err)
  }
  now := time.Unix(txTimestamp.Seconds, 0).UTC()
  if grant, ok := doc.ACL[userID]; ok &&
      grant.Status == StatusActive {
    if grant.ExpiresAt == "" {
      return "true", nil
    }
    exp, err := time.Parse(time.RFC3339,
        grant.ExpiresAt)
    if err == nil && now.Before(exp) {
      return "true", nil
    }
    grant.Status = StatusExpired
    // NOTE: No state write-back occurs here. Expiry is
    // evaluated dynamically on every invocation to avoid
    // MVCC read-write conflicts under concurrent load;
    // StatusExpired is only persisted later by the
    // asynchronous CleanExpiredTokens background job.
    return "false", nil
  }
  if doc.OwnerOrg == userOrg {
    return "true", nil
  }
  // Branch 4-5: org-prefix grant
  if grant, ok := doc.ACL["ORG:"+userOrg]; ok &&
      grant.Status == StatusActive {
    if grant.ExpiresAt == "" {
      return "true", nil
    }
    if exp, err := time.Parse(time.RFC3339,
        grant.ExpiresAt); err == nil &&
        now.Before(exp) {
      return "true", nil
    }
  }
  return "false", nil
}
```

On the evaluated document-download path, Fabric unavailability causes HTTP 503 denial and no database ACL fallback. Other Fabric-gateway paths follow the same fail-closed pattern by design but were not independently outage-tested. The service immediately logs a `FABRIC_OUTAGE_ACCESS_DENIED` audit event containing the failure reason and throws a `ServiceUnavailableException`, returning HTTP 503 to the caller. This prevents new document releases through this path during the tested Fabric outage and prevents a malicious actor from using a Fabric disruption to bypass the chaincode authorization layer. Listing 5 (Appendix Appendix A) shows the two-layer enforcement in the download path. If the requester has no firm or MSP binding, the

service throws `AccessDeniedException` before invoking the Fabric gateway, ensuring the download path is fail-closed unconditionally for unauthenticated or incomplete principal bindings.

The complete two-layer enforcement path is summarized in Fig. 13, and Algorithm 2 formalizes the capability-grant construction.

Algorithm 2 Capability-Based Access Grant

Require: Document ID d, subject s, permission p, conditions $\mathcal{C}$
Ensure: Capability token T or $\perp$
$t_{exp} \leftarrow t_{now} + \mathcal{C}.\Delta t$ {$\Delta t = \infty$ if no expiry}
$S_{scope} \leftarrow \text{Parse}(\mathcal{C}.scope)$
$T \leftarrow (d, s, p, t_{exp}, S_{scope})$
$\sigma_T \leftarrow \text{ECDSA-P256.Sign}(\mathcal{H}(T), sk_{grantor})$
$\mathcal{R} \leftarrow \{\mathcal{C}.\text{triggers}\}$
Blockchain.Store$(\langle T, \sigma_T, \mathcal{R} \rangle)$
emit `ACCESS_GRANTED`$(d, u_{grantor} \rightarrow s)$
return T

3.4. Audit Architecture

The system maintains two independent audit stores by design. Their independence is a deliberate security property, but the two stores provide different guarantees.

Fabric ledger (primary, cryptographic). Every significant event is recorded by the `LogAuditEvent` chaincode as an immutable ledger transaction. Any consortium peer can query the history of any document key using `GetHistoryForKey`, which returns a chronologically ordered, hash-linked sequence of state transitions. The Fabric block hash chain provides cryptographic tamper-evidence natively; the application does not maintain a separate SHA-256 chain over on-chain records because doing so would be redundant with Fabric's own block structure.

PostgreSQL `audit_log` (secondary, operational). A separate append-only table records all events for fast dashboard queries. An INSERT-only enforcement trigger at the database level raises an exception on any UPDATE or DELETE attempt regardless of application privileges. A TRUNCATE-prevention trigger closes the PostgreSQL DDL bypass gap, and TRUNCATE privilege is revoked from all application roles at the schema level. Listing 6 (Appendix Appendix A) shows both triggers. These triggers are an *operational convenience* that raises the bar against accidental or application-layer modification; they are not a security defense against a database administrator with superuser access. A superuser can drop triggers, alter the schema, or bypass the SQL engine entirely via direct file access, rendering the trigger

layer ineffective against the Malicious DB Administrator adversary class defined in Table 5. The security guarantee against that adversary relies entirely on the Fabric ledger, which the database administrator cannot access or modify: any consortium peer can independently verify the complete audit trail via `GetHistoryForKey` without trusting the application server or its database [10].

Experiment 4 (Section 6.4) measures PostgreSQL `audit_log` query P50 at 3.9 ms for 1,000 events on Linux `x86_64`, and Experiment 7 (Section 6.7) measures Fabric `GetHistoryForKey` at 135.5 ms P50 for a 107-entry document history. The two stores serve complementary roles: PostgreSQL supports low-latency operational dashboards, while Fabric provides cryptographic non-repudiation and independent verification by any consortium peer without trusting the application server.

4. Governance, Security, and Consensus

4.1. Operational Threat Model

This section defines the operational security assumptions under which the framework is evaluated. The goal is not to present a new cryptographic proof, but to make explicit which adversaries are in scope, which trust boundaries are enforced by Fabric, and which risks remain in the measured prototype. The analysis covers consortium governance assumptions, Raft crash-fault-tolerant ordering limits, endorsement-policy limits, application and browser compromise, identity-management boundaries, on-chain metadata leakage, and residual risks requiring production hardening.

The deployment model is a permissioned legal consortium consisting of known, legally bound institutions such as law firms and a regulator. Participants are authenticated through Fabric MSPs, and state-changing transactions are validated by endorsement policies before being ordered and committed. Under normal operation, document access is evaluated through the `CheckAccess` chaincode before protected material is released. On the evaluated document-download path, Fabric unavailability causes HTTP 503 denial and no database ACL fallback. Other Fabric-gateway paths follow the same fail-closed pattern by design but were not independently outage-tested. The threat model therefore identifies the custodial Fabric identity and `localStorage`-based key handling as the primary residual prototype limitations, not a database fallback.

Bounded adversary assumption. Two independent bounds apply: (1) *CFT orderer bound*: with a 3-node Raft cluster ($n = 3$), the crash-fault tolerance bound is $f \leq \lfloor (n-1)/2 \rfloor = 1$; the cluster tolerates failure of any single ordering node while maintaining consensus progress [13]. This bound applies to node crashes, not Byzantine behavior. (2) *Endorsement policy bound*: when a deployment requires `AND('LawFirmAMSP.peer', 'RegulatorMSP.peer')`, an adversary cannot satisfy that policy by controlling a single organization, because both organizations' peer endorsement is required independently. These bounds are independent: orderer quorum governs block formation; endorsement policy governs state write validity. The adversary may be adaptive but cannot break the cryptographic primitives (AES-256-GCM, ECDSA P-256, SHA-256) under their standard hardness assumptions.

The prototype's ECIES grants are ledger-recorded but depend on an off-chain identity-to-key binding: PostgreSQL stores each user's `pk_ecies` and `pk_sign`. Until these public keys are anchored by a `RegisterIdentity`-style ledger transaction, a database or API adversary who substitutes `pk_ecies` before a grant is formed can cause future `GrantAccess` operations to wrap k_{enc} to an attacker-controlled key. Fabric would then commit a cryptographically valid transaction whose recipient key binding is semantically wrong. This threat is distinct from direct ledger tampering and is treated as a highest-priority production hardening requirement.

Table 5 identifies seven adversary classes relevant to the legal consortium deployment.

Table 5: Operational Threat Model: Adversary Classes and System Defenses. This is a structured adversary analysis, not a formal cryptographic security proof. Residual risk column identifies properties that require governance controls or hardware beyond the current prototype.

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Honest-but-curious peer	Can observe all transactions on joined channels; can read world state; cannot modify ledger or forge signatures	Confidentiality of document content and access patterns	Documents stored as AES-256-GCM ciphertext in private IPFS swarm; CIDs and hashes only on-chain; PDCs limit metadata to authorized peers [34]	Channel members still see CIDs, timestamps, accessor MSP identities, PDC hashes, and plaintext hash $\mathcal{H}_D$, leaking frequency patterns and enabling candidate-document confirmation via local `SHA-256`. The prototype uses organization-level membership (`doc.OwnerOrg == userOrg`) as first-pass control; per-user intra-organization ACLs and moving $\mathcal{H}_D$ to a PDC or keyed commitment $\mathrm{HMAC}_k(D)$ remain production hardening.

Table 5 (continued)

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Malicious DB administrator	Full write access to PostgreSQL; can modify application tables; cannot access Fabric peers or IPFS directly	Integrity of access control records and audit log	Under normal operation, access control is evaluated by chaincode (Layer 2). During Fabric unavailability, the system fails closed (returning HTTP 503). The `audit_log` row-level and statement-level triggers block accidental or application-layer modification and TRUNCATE attempts (Listing 6)	Triggers provide operational integrity only, not cryptographic protection from a PostgreSQL superuser. The DB admin cannot directly alter Fabric ACL or audit history, and Fabric outages deny access; however, replacing `pk_ecies` can make later grants wrap k_{enc} to an attacker key while the ledger records an apparently valid grant. Null firm/MSP now raises `AccessDeniedException` before the Fabric gateway; no default organization is assumed (Listing 5).

Table 5 (continued)

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Compromised application server / API operator	Controls the Spring Boot service, API-layer RBAC middleware, REST gateway, and operational database access; may attempt to bypass chaincode, misuse server-managed Fabric MSP credentials, serve tampered frontend JavaScript to intercept client-side operations, cache or forward IPFS ciphertext after legitimate delivery, or abuse MSP identity bindings before ledger commitment	Authorization integrity, transaction attribution, confidentiality in prototype plaintext-handling paths	In the honest-code normal path, document release invokes `CheckAccess` against ledger-visible ACL state before protected material is returned. Fabric provides an independently verifiable record for ledger-mediated grants, revocations, and audit events; Fabric outage events log as `FABRIC_OUTAGE_ACCESS_DENIED`; ledger-mediated user-level document signatures bind the signing key to $\mathcal{H}_D$, $\mathcal{H}_C$, and id_D	A fully compromised API operator can bypass application logic, misuse custodial MSP credentials, expose plaintext via malicious frontend delivery before browser encryption, abuse off-chain access paths, or substitute recipient keys before grant creation. Fabric still makes ledger-mediated state changes auditable; production requires browser-only encryption, per-user Fabric identities, ledger-anchored public keys, and strict service hardening.
Compromised browser / XSS attacker	Executes malicious JavaScript in the user's browser session; can observe passwords or decrypted private keys during unlock/use; can initiate actions as the logged-in user; may tamper with frontend code before encryption or signing	Client-side key confidentiality, document confidentiality before encryption, and authenticity of user-initiated actions	Private keys are stored encrypted in browser `localStorage` under a PBKDF2-derived AES-GCM wrapping key; plaintext and signing operations are intended to occur in the browser; server-side audit records and Fabric logs make submitted actions traceable	Encrypted `localStorage` resists passive storage theft, not active browser compromise. XSS or malicious frontend code can capture passwords, plaintext, or keys during use; production requires CSP, supply-chain controls, WebAuthn/FIDO2 or HSM-backed keys, and hardened frontend delivery.

Table 5 (continued)

Adversary	**Capabilities**	**Property threatened**	**System defense**	**Residual risk**
Compromised IPFS node	Can serve modified or stale ciphertext; cannot forge SHA-256 hashes	Document integrity and availability	SHA-256 hash of plaintext stored on-chain at registration; any tampered ciphertext produces a hash mismatch detectable by any authorized client on download; the design uses a private two-node swarm/cross-pinning configuration	Availability still depends on pinning policy and node health. If both IPFS nodes are compromised or unavailable, content may be unavailable; production requires $\geq$3 pinning nodes or an SLA-backed pinning service.
Orderer collusion (1-of-3 Raft)	A single compromised orderer can observe all unordered transactions; cannot unilaterally reorder or censor if the 2-of-3 quorum continues to function	Transaction ordering integrity and liveness	3-node Raft: any single orderer failure is tolerated while consensus continues; Fabric's execute-order-validate pipeline means peers independently validate every block regardless of orderer behavior [10, 12]	Two colluding orderers (2-of-3) can censor or delay transactions; CFT does not prevent Byzantine ordering. Fabric v3.0 SmartBFT [43, 44] is the production path for high-threat deployments.
On-chain metadata leakage	Channel members can observe transaction timestamps, invoking MSP identity, and CIDs even for PDC-protected data	Access pattern privacy	PDCs store sensitive metadata in a private collection; only the collection hash is written to the main ledger [34]. A channel member observing transaction frequency and CID patterns over time can infer which documents are accessed frequently, potentially revealing case activity patterns without decrypting content	Traffic analysis can reveal access patterns without document content. Zero-knowledge proofs, batching, cover traffic, or private access protocols are future work to reduce leakage (Section 7.2).

Table 6: Compact security argument sketch. Residual risks are not eliminated by the prototype.

Claim	Argument	Residual risk
Fabric audit history cannot be silently altered by a DB admin	Fabric is append-only across endorsing peers; PostgreSQL triggers are only operational support [10].	DB superuser can alter off-chain identity data; production path anchors public keys on-chain.
IPFS ciphertext cannot be modified undetectably	Plaintext hash $\mathcal{H}_D$ is anchored on-chain; altered ciphertext causes client-side verification failure [11].	Content availability still needs $\geq$3 pinning nodes or an SLA-backed pinning service.
Fabric outage fails closed	`CheckAccess` is on the document-release path; Fabric failure returns HTTP 503 and logs `FABRIC_OUTAGE_ACCESS_DENIED`.	None observed on download path; upload, grant, and revoke paths are fail-closed by design but not independently validated.
API compromise remains residual	ECDSA binds document commitments to user keys, and Fabric records ledger-mediated events; prototype MSP credentials remain server-managed.	Compromised server can abuse custodial MSP credentials; production requires per-user Fabric identities.
Browser/XSS compromise remains residual	Keys are PBKDF2-protected in `localStorage`; signing/decryption are intended to occur client-side.	Active XSS can capture keys or plaintext; mitigation requires WebAuthn/FIDO2, CSP, and supply-chain controls.
2-of-3 Raft orderer collusion remains residual	Peers validate blocks independently after ordering [10]; governance reduces but does not cryptographically prevent collusion.	CFT does not stop Byzantine ordering; SmartBFT is the documented high-threat production path [43, 61].

Malicious Fabric peer clarification. Table 5 separates honest-but-curious peer observation from active peer compromise. A malicious Fabric peer may run altered local software, leak channel or world-state metadata, withhold endorsements, return misleading query responses, or collude with other organizations. It cannot by itself commit invalid ledger state when a transaction requires multi-organization endorsement, because committing peers validate endorsement signatures, read-write sets, MVCC versions, and chaincode-definition consistency before updating world state [10, 12]. Residual risks remain if enough endorsing organizations collude, if clients trust a single peer for query results, or if channel metadata itself is sensitive; production deployments therefore require multi-organization endorsement policies, peer monitoring, query-provenance checks where appropriate, and governance sanctions.

Table 6 summarizes the principal security claims, their architectural support, and the residual risks left by the current prototype.

The framework uses Raft, a Crash Fault Tolerant (CFT) protocol. CFT tolerates node failures but not Byzantine orderer behavior. This choice is a *performance trade-off*, not a cryptographic security guarantee. Legal accountability reduces the probability of collusion among orderers operated by distinct, legally-bound institutions, but provides no cryptographic bound

against Byzantine behavior: a sufficiently motivated or compromised orderer set can still violate CFT's safety assumptions. As noted in the threat model table, 2-of-3 orderer collusion remains a residual risk under CFT, and the current Raft configuration is a *prototype constraint* rather than a framework recommendation. For deployments in which Byzantine fault tolerance is required, the SmartBFT ordering service introduced in Hyperledger Fabric v3.0 is the recommended path [43, 44, 61, 10]. The current prototype uses Fabric v2.4 Raft; migration to SmartBFT does not require changes to chaincode or application logic.

SmartBFT migration boundary. Fabric's BFT configuration guide provides operational guidance for BFT orderer deployments [61]. Migrating from Raft to BFT requires a Fabric v3.0-or-later network, channel capability `V3_0` or later, and a BFT orderer set sized as $3f + 1$ for the desired Byzantine fault bound. SmartBFT can reduce throughput because Byzantine agreement requires additional consensus messages and a larger minimum orderer set than a 3-node Raft cluster. The evaluated prototype did not implement or measure SmartBFT, so this paper does not project SmartBFT throughput from the Raft measurements. BFT ordering is treated as production hardening and future work for high-threat deployments requiring protection against Byzantine orderer collusion.

Cryptographic security properties. The threat model above is a structured operational analysis, not a formal cryptographic proof with security games or reduction arguments. The individual primitives are used within standard design assumptions: AES-256-GCM provides authenticated encryption under nonce-uniqueness assumptions, as specified in NIST SP 800-38D [48]; ECDSA P-256 provides digital-signature unforgeability under the elliptic curve discrete logarithm assumption, per FIPS 186-5 [16]; and the ECIES-style ECDH + KDF + AES-GCM construction follows the standard hybrid-encryption pattern for wrapping document keys [50]. The analysis does not claim a new reduction for the composed protocol, and a full compositional security proof is left as future work.

4.2. Security Architecture

Security is implemented through a multi-layered, defense-in-depth approach that separates the intended framework boundary from the measured prototype boundary. The framework design places encryption, signing, and key unwrapping in the browser and uses Fabric as the independently verifiable authority for ACL and audit state. The measured prototype preserves the

main ledger-visible checks and enforces strict fail-closed access control, but still includes server-managed Fabric MSP credentials and `localStorage`-based signing keys as residual prototype limitations.

4.2.1. Cryptographic Framework

The system employs a three-tier hybrid cryptographic approach. At the file encryption layer, documents are encrypted client-side in the framework design [14]. For secure key sharing, the AES document key is wrapped using ECIES-style P-256 via the browser WebCrypto API [15], producing a 125-byte token that is 51.2 % smaller than RSA-OAEP 2048 (256 bytes), as measured in Experiment 6 (Section 6.6). At the infrastructure layer, Hyperledger Fabric uses ECDSA (NIST P-256) for transaction signing and MSP identity validation [33, 16].

4.2.2. Identity Management

The system's identity management design is aligned with the NIST SP 800-63 series. NIST SP 800-63-4 supersedes SP 800-63-3 (withdrawn 2025-08-01) [58, 62]. In the current prototype, authentication is limited to username/password login combined with signed JWTs [60]. The architecture is compatible with migration toward SP 800-63-4-aligned AAL2/AAL3 deployments, but the prototype evaluates only username/password plus JWT authentication. A production deployment can integrate an external identity provider enforcing phishing-resistant MFA, WebAuthn/FIDO2, or hardware-backed authenticators while leaving the RBAC and ACL layers in Section 3 unchanged. This upgrade path requires no chaincode modification.

4.3. Consensus Mechanism and Validation

The framework employs Hyperledger Fabric's Raft consensus protocol [13, 12]. The Raft ordering service is configured as a cluster of three ordering nodes operated by distinct, audited consortium members: one node operated by the regulator and two nodes operated by separate law-firm organizations. This configuration is crash-fault tolerant: with $n = 3$, Raft tolerates one crash fault ($f = 1$) while maintaining consensus progress. This governance claim refers to operational control of the ordering nodes. A production deployment should also avoid a single administrative trust point for orderer identity issuance, either by using member-controlled orderer MSP/CA administration or by placing any shared ordering CA under multi-party consortium control. If a prototype or repository configuration represents the orderers under a shared

`OrdererOrg` CA, that should be interpreted as a deployment simplification and residual governance limitation rather than as the production governance model. In the current prototype, the Regulator organization participates as an ordering node and a passive channel peer; application-layer regulatory audit interfaces, dedicated read-only compliance views, and regulator-specific endorsement policies (e.g., `AND('LawFirmAMSP.peer','RegulatorMSP.peer')` for high-value asset transfers) are deferred to future work (Section 7.2). It does not provide Byzantine-fault tolerance and does not protect against malicious 2-of-3 orderer collusion. No single institution unilaterally controls transaction ordering under the stated consortium-governance assumption, provided that both orderer operation and orderer-identity administration are governed across consortium members. Every transaction undergoes a multi-stage validation process before being committed to the ledger [10, 35]. For example, a deployment may require endorsement from one peer of the submitting organization for document upload (`OR('LawFirmAMSP.peer')`) and endorsements from both the owner's organization and the regulator for access grants (`AND('LawFirmAMSP.peer', 'RegulatorMSP.peer')`). These policies are illustrative unless matched by the deployed channel and chaincode configuration. A new chaincode definition can only be committed after a sufficient number of consortium member organizations have approved it to satisfy the channel's LifecycleEndorsement policy, ensuring smart-contract upgrades require multi-organization agreement [63].

5. Prototype Implementation

5.1. Prototype Scope

The paper presents a target hybrid framework; the current repository implements a proof-of-concept prototype. Table 7 documents the gaps between framework design goals and current prototype status.

In the user-facing frontend upload workflow, the prototype implements the confidentiality boundary: plaintext remains on the user device, browser-native cryptography performs strict client-side encryption [14, 15], and AES-256-GCM protects each payload before IPFS storage. The benchmark harness used in the performance experiments isolates the REST-gateway, Fabric, database, and IPFS paths by submitting opaque upload payloads that represent the IV-prepended ciphertext accepted by the server; therefore, Experiments 1–3 do not measure browser-side encryption time. Access control is also on the normal download path. Each download invokes the `CheckAccess` chaincode before protected material is released.

When Fabric is unreachable, the prototype fails closed. It records a `FABRIC_OUTAGE_ACCESS_DENIED` event, returns HTTP 503, and does not bypass the blockchain authorization layer [47]. The prototype implements a 2-node private IPFS swarm with cross-pinning and logical deletion [64, 65].

Fabric transaction-level attribution remains custodial because the prototype uses server-managed Fabric MSP credentials. User-level ECDSA document signatures still bind the signing key to the exact plaintext content ($\mathcal{H}_D$), the exact IPFS storage payload ($\mathcal{H}_C$), and the document identifier (id_D). Binding upload timestamp and owner metadata is a production hardening item, so these signatures should not be interpreted as per-user Fabric transaction non-repudiation.

Table 7: Framework Design vs. Prototype Implementation. Y = implemented; Y* = implemented with a known availability or governance caveat (see Production path column); P = partial; N = not yet implemented. Production path column identifies the engineering step required to close each gap. †The ECDSA signing/verification mechanism is implemented and binds the signing key to $\mathcal{H}_D$, $\mathcal{H}_C$, and id_D; binding upload timestamp and owner metadata is a production hardening item. Fabric transaction-level attribution remains custodial because the prototype submits Fabric transactions through server-managed MSP credentials (Section 5.1).

Feature	**Framework design**	**Prototype**	**Production path**
Client-side encryption	Browser AES-256-GCM; plaintext never reaches server	Y	Already implemented
ECDSA document signatures	Browser ECDSA P-256 sign; server verifies before Fabric anchoring	Y†	Mechanism implemented and binds $\mathcal{H}_D$, $\mathcal{H}_C$, and id_D; Fabric transaction-level non-repudiation requires per-user Fabric identity (Section 5.1)
ECIES-style P-256 key wrapping	125-byte token; browser wraps k_{enc} under recipient pk_{ecies}	Y	Already implemented
Fail-closed Fabric invocation	`CheckAccess` evaluated on every download; service denies access on Fabric outage	Y	Implemented

Table 7 (continued)

Feature	**Framework design**	**Prototype**	**Production path**
Per-user and org-prefix ACL evaluation	`CheckAccess` evaluates per-user `ACL[userID]` grants (Branches 1–2), falls back to `doc.OwnerOrg == userOrg` for owner-organization access (Branch 3), and evaluates org-prefix grants under `ACL["ORG:"+org]` (Branches 4–5), as shown in Listing 1.	Y	Implemented
Fine-grained intra-organization per-user ACL	Remove `doc.OwnerOrg == userOrg` owner-org fallback; require explicit per-user grants for all principals, including intra-organization peers	P	Prototype currently grants access to any member of the document owner's organization; per-user grants are enforced only for cross-organization access. Production hardening item.
Time-bounded capability tokens	On-chain expiry enforced in chaincode; expired grants denied by `ExpiresAt`; expiry write-back omitted on read path to prevent MVCC conflicts; asynchronous `CleanExpiredTokens` cron-job handles ledger cleanup	Y	Implemented; concurrent expiry write-back behavior treated as production hardening

Table 7 (continued)

Feature	**Framework design**	**Prototype**	**Production path**
3-node Raft ordering	Cluster across 3 distinct consortium members (2 firms + regulator)	Y	Already implemented; CFT property follows from Raft configuration
2-node IPFS swarm	Replicated private pinning across 2 nodes; availability depends on pinning policy, node health, and network reachability	Y*	Implemented as private two-node replication; ≥3 nodes or managed pinning recommended for production SLA
Per-user Fabric identity	Per-user X.509 certificate issued by Fabric CA; HSM-backed	N	Per-user Fabric CA enrollment; HSM wallet integration
Hardware-backed signing key	WebAuthn/FIDO2 or TPM; signing key never in localStorage	N	WebAuthn/FIDO2 integration replacing localStorage
Key rotation on revocation	Browser re-encrypts k_{enc} for remaining authorized users	P	Notification implemented; browser re-encryption requires owner action; proxy re-encryption is the interim path
TOTP MFA recovery codes	Recovery codes generated at enrollment; admin re-enrollment flow	N	TOTP recovery code generation at enrollment
Hardware TLS attestation	TPM/HSM-backed TLS certificates on Fabric peers	N	Replace software certificates with hardware-attested TPM certs

Table 7 (continued)

Feature	Framework design	Prototype	Production path
Fabric PDC storage for wrapped-key tokens	Wrapped-key tokens may be stored in PDCs to reduce key-token metadata exposure	N	Prototype stores ECIES-wrapped tokens as encrypted Fabric world-state values indexed by document and recipient; PDC storage is production hardening.

The "Per-user and org-prefix ACL evaluation" row refers to implemented cross-organization grant evaluation in the access-control logic, whereas "Intra-organization access control" refers to finer-grained per-user enforcement within the same owner organization. The latter remains partial because owner-organization fallback access can bypass strict intra-organization per-user granularity.

5.2. Technology Stack

The technology stack was selected to satisfy enterprise-grade requirements for security, scalability, and modularity. Hyperledger Fabric v2.4 provides the permissioned blockchain core, including private data collections and pluggable consensus [10]. CouchDB 3.2 serves as the peer state database for rich JSON queries over document metadata and ACL structures [66]. Chaincode is implemented in Go for native Fabric ecosystem support [63], and IPFS (Kubo, Dockerized) provides content-addressed off-chain encrypted storage [11].

The application backend is a Spring Boot 3.2.5 REST API running on Java 21 (OpenJDK). This stack was selected for `@PreAuthorize` Spring Security integration on the Layer 1 RBAC path and for native Hyperledger Fabric Java SDK support [67]. Off-chain identity and audit data are stored in PostgreSQL 16 [68], and the web frontend is implemented in React. The evaluation network is containerized with Docker and Docker Compose.

Experiment 6 used Node.js WebCrypto as a same-API fallback because browser automation was unavailable. PBKDF2-SHA256 at 600,000 iterations measured 100.44 ms P50 over 10 trials, and ECIES-style P-256 reduced the wrapped-key token from 256 bytes under RSA-OAEP 2048 to 125 bytes. The production stack uses Java 21. The prototype manages user MSP wallets custodially on the server under a shared administrative identity; production deployment would shift to per-user X.509 enrollment with non-custodial browser-based signing [69, 45].

5.3. Blockchain Network Topology

The repository contains configuration artifacts for a larger six-peer-organization topology: five representing law firms (Firm-A to Firm-E) and one representing a regulator, plus a separate ordering organization, as depicted in Fig. 14. The separate `OrdererOrg` representation is a prototype/repository identity layout. If the orderers are issued under one shared ordering CA, that CA remains a residual administrative trust point unless its administration is controlled by multiple consortium members. The production governance model in Section 4.3 therefore requires member-controlled orderer identity administration or multi-party control over any shared ordering CA. The measured prototype deployment used a shared `OrdererOrg` CA to issue the three orderer identities across the three-organization configuration described in the evaluation: LawFirmA, LawFirmB, and RegulatorOrg, with three orderers and three peers; this repository-level simplification does not reflect the member-controlled CA governance model in Section 4.3, which is the production target. The ordering service is a 3-node EtcdRaft cluster operated by three distinct consortium members (one regulator node and two law-firm nodes), consistent with the governance configuration described in Section 4. This configuration is crash-fault tolerant under Raft assumptions, tolerating one crash fault in a three-node cluster [12]. Encrypted document payloads are stored off-chain in a private 2-node IPFS swarm with cross-pinning; this configuration provides replicated private pinning, but availability still depends on pinning policy, node health, and network reachability. It should not be interpreted as broad public-IPFS decentralization [70].

Consequently, all performance claims in Section 6 refer to the measured three-organization configuration, not the larger repository topology. This subset understates realistic gossip and endorsement overhead compared to the full deployment; throughput numbers should be treated as an upper bound for the measured 3-org configuration.

5.4. Chaincode Design and Functionality

The core business logic is encapsulated in the `legalcc` chaincode, which implements: `RegisterDocument` (anchors document hash, CID, and ECDSA signature as a ledger asset); `GrantAccess` and `RevokeAccess` (manage capability tokens with optional expiry in the on-chain ACL map); `CheckAccess` (evaluates the on-chain ACL at query time with time-bounded token enforcement; expiry write-back is omitted on the read path to prevent MVCC read-write conflict crashes, as documented in Listing 1); `GetHistoryForKey`

(returns the full hash-linked transaction history for any document key); and `CleanExpiredTokens` (asynchronous administrative function that formally deprecates expired grants in world state, invoked periodically via a background cron-job; not required for authorization correctness, since `CheckAccess` evaluates expiry dynamically at query time) [71]. Expiry write-back is not relied on for authorization correctness: expired grants are denied based on their `ExpiresAt` value. Concurrent write-back behavior for the same expired grant was not isolated in the experiment campaign and is treated as a production hardening issue.

5.5. Demonstration of Core System Workflows

5.5.1. Workflow 1: Granting Access Permissions

Fig. 15 shows the access grant sequence. The document owner invokes `GrantAccess` through the frontend; the application server relays to the chaincode, which writes the time-bounded capability token to the ACL map in world state and emits an `ACCESS_GRANTED` audit event.

5.5.2. Workflow 2: Verifying Access and Retrieving a Document

Fig. 16 illustrates the document retrieval sequence. The application server enforces Layer 1 Spring Security RBAC [47], then invokes `CheckAccess` chaincode (Layer 2). If both layers approve, the server retrieves the IPFS ciphertext and returns it with the ECIES-wrapped key token for client-side decryption.

5.5.3. Workflow 3: Activity Flow for Document Upload

Fig. 17 presents the upload activity diagram. In the framework design, encryption occurs client-side before the application server forks two parallel threads: IPFS storage and blockchain anchoring. A join node ensures both must complete before confirmation is returned. The measured prototype fully implements the client-side encryption boundary; the parallel fork/join thread model in this diagram is a framework-design detail not independently validated in the experiment campaign.

5.6. User Interface and Functional Showcase

The prototype exposes the full workflow through a web interface: registration with role selection, a navigation dashboard, card-based case management, and document dialogs that surface the content identifier, Fabric transaction identifier, integrity hash, and wrapped-key reference for each file, together

with controls for downloading the encrypted object or a locally decrypted copy. Because auditability is the central claim of this work, Fig. 18 shows the two transparency views. The ledger explorer (Fig. 18a) groups recent entries by block and displays transaction IDs, channels, timestamps, transaction types, and document references. The audit-log view (Fig. 18b) lists application events with timestamps and expanded metadata so administrators can review user activity from within the prototype interface. The remaining interface screenshots are provided as supplementary material.

6. Performance Evaluation

Nine experiments evaluate RQ1–RQ3. Measurements were collected on three platforms: a Windows 11 workstation (Ryzen 7 5700X, Java 17 Temurin, Node.js 24), an Apple M1 MacBook under Rosetta 2 emulation (macOS, Java 21 Temurin, Node.js 22), and a native Linux x86_64 server (Intel Core Ultra 5 125H, Ubuntu 22.04, 8 GB RAM, NVMe SSD, Java 21 OpenJDK, Node.js 18). All authoritative figures are from Linux; M1 and Windows results are auxiliary cross-platform checks where reported.

The evaluated Fabric network used three peer organizations (LawFirmA, LawFirmB, RegulatorOrg) on a shared `legalchannel`, each backed by CouchDB 3.2. The ordering service was a 3-node EtcdRaft cluster across three distinct consortium members. This configuration tolerates one crash fault under CFT assumptions, but the experiment campaign did not evaluate Byzantine ordering behavior [12].

Unless otherwise stated, `BatchTimeout` was set to 2 s and `BatchSize.MaxMessageCount` to 500. Sustained throughput was measured with a 60 s fixed-duration closed-loop load generator; a fixed-count harness was used as a latency cross-check. A custom Node.js harness targeted the Spring Boot REST gateway with a 20 % write / 80 % read mix. Each data point is the mean of five independent runs after a discarded warm-up round, following standard Hyperledger Fabric benchmarking practice [72, 73]. Table 8 summarizes the configurations.

Direct numerical comparison with prior systems is not possible because, to the best of the authors' knowledge, the systems in Table 1 do not report REST-gateway throughput under concurrent load with a documented traffic mix [24, 25, 26, 27, 28]. The PostgreSQL-only baseline is therefore used as a reproducible upper-bound reference [74].

For upload-path experiments, the load generator submits opaque byte payloads representing the IV-prepended ciphertext that the server receives in

the browser workflow. Consequently, Experiments 1– 3 isolate REST-gateway, Fabric, database, and IPFS costs; browser-side encryption is measured separately in Experiment 6.

6.1. Experiment 1: Scalability Under Concurrent Load

This experiment varies concurrent client load from 50 to 600 and measures committed throughput in both Fabric mode and a PostgreSQL-only baseline, to answer RQ3. Writes were issued as `POST /documents/upload` requests and reads as `GET /wrapped-key`, with a 10 s per-request timeout and five repetitions per concurrency level.

The framework REST gateway with Fabric sustained approximately 68–70 TPS across the measured 50–600 client range, remaining about 4.1 to 4.2× above the synthetic baseline used as an illustrative capacity-planning assumption [1, 19] for a 1,000-user firm performing one document action per minute per user and showing zero transaction errors up to 400 clients. Throughput is invariant with respect to concurrency: as load rises, per-request latency grows while committed throughput holds flat, the signature of a system gated by the fixed 2 s Raft batch window rather than by compute or concurrency. CPU utilization stayed below 23 % throughout confirming the bottleneck is the Raft orderer `BatchTimeout=2s`, not compute. The PostgreSQL-only baseline peaked near 279 TPS in the stable region. Beyond that point, the closed-loop load generator saturates, so the shaded region in Fig. 19 is treated as a client-side limit rather than a database-server failure.

Table 8: Experimental Hardware Configurations

Parameter	**Windows**	**M1 (Rosetta)**	**Linux (auth.)**
OS	Win 11 Pro	macOS amd64 emu.	Ubuntu 22.04
CPU	Ryzen 7 5700X	Apple M1	Core Ultra 5 125H
RAM	16 GB	16 GB	8 GB
Java	17 Temurin	21 Temurin	21 OpenJDK
Fabric	2.4.x	2.4.x	2.4.x
Orderer	3-node Raft	3-node Raft	3-node Raft
Role	Auxiliary	Exps. 1–4	Exps. 1–9

To evaluate operational viability, we establish a synthetic steady-state baseline $1000/60 \approx 16.7$ TPS. The sustained ≈68–70 TPS throughput provides a 4.2× safety margin over this average.

Three considerations bound this interpretation. First, the experiment used the measured 3-organization deployment rather than the larger repository topology. Larger deployments will incur additional gossip, endorsement, and resource overhead, so the measured throughput is an upper bound for the evaluated configuration rather than a guarantee for all consortium sizes.

Second, `BatchTimeout` is tunable. Reducing it from 2 s to 500 ms raises the highest measured throughput to 193.0 TPS under the evaluated three-organization deployment, an 11.6× margin over the 16.7 TPS baseline without architectural change [12].

Third, enterprise traffic is bursty. An aggressive 10× burst factor raises the target to ≈167 TPS; Experiment 8 (Section 6.8) shows that the 500 ms setting reaches 193.0 TPS, a 15.6% margin above that peak target [75]. The throughput cost of blockchain consensus is therefore a configuration trade-off, not an architectural barrier.

6.2. Experiment 2: Function-Level Operation Latency

This experiment isolates per-operation latency for `CheckAccess` (read) and `RegisterDocument` (write) in Fabric and DB-only modes, providing the direct answer to RQ2 and resolving the baseline inconsistency cited in prior review.

`CheckAccess` latency was measured with a Node.js `performance.now()` timer under sequential single-client load. Twenty warm-up calls were discarded before 100 measured samples; the same protocol was applied to the DB-only baseline on the same hardware. Warm-up matters because JIT compilation can modestly overstate framework cost before the JVM has stabilized.

`RegisterDocument` latency was measured end-to-end through the Spring Boot Fabric gateway. The write path was verified to commit on-chain: each call returned a real `fabricTxId`, and `GetDocumentHistory` confirmed the same identifier. As with the read-path test, 100 measured samples followed 20 warm-up calls.

The write path is dominated by ordering latency. `BatchTimeout` accounts for ≈96 % of the 2,084 ms total, and the independent Experiment 3 commit-latency constant ($2{,}132 \pm 20$ ms across all file sizes) is consistent with orderer processing dominating the path. Experiment 2 and Experiment 3 use inde-

pendent sample sets and different harness boundaries, so their values are reported as consistent observations rather than additive components.

On native Linux x86_64 with JVM warm-up, Fabric access checks are operationally indistinguishable from the DB-only path across the full latency distribution (Table 9). Inter-quartile ranges overlap, P99 differs by only 2.3 ms, and a two-tailed Mann-Whitney U test finds no statistically significant difference ($U = 4500.0$, $p = 0.22$, $n = 100$ per path). Both paths remain below the 50 ms interactive threshold at all measured percentiles, including P99 (Fabric 15.5 ms, DB-only 13.2 ms). The slightly higher Fabric P99 is consistent with occasional gateway scheduling jitter and does not affect the threshold conclusion.

A standalone PostgreSQL-only write baseline was not collected because the framework has no write path that bypasses Fabric. Measuring isolated PostgreSQL inserts would omit the Fabric SDK endorsement round-trip and orderer batch costs that are structurally part of every document registration. The absolute `RegisterDocument` latency (2,084 ms P50) is therefore evaluated directly against the 5 s SLO. Experiment 1 corroborates this interpretation: under the 68-70 TPS Fabric load, PostgreSQL operated at approximately 25% of its independently observed write-capacity reference (≈279 TPS), and CPU utilization remained below 23%. Cross-platform results also support BatchTimeout dominance: the Apple M1 session measured 2,147 ms, within 3 % of the Linux figure, as expected when the orderer timeout accounts for ≈96 % of total write latency.

Table 9: Read-path latency distribution: `CheckAccess` via Fabric gateway vs. DB-only ACL lookup, warmed runs ($n = 100$ each, Linux x86_64).

	Fabric	**DB-only**
Mean (ms)	7.293	7.548
Std dev	2.352	2.092
P25 (ms)	5.698	5.895
P50 (ms)	6.510	7.162
P75 (ms)	8.458	8.753
P95 (ms)	10.871	11.283
P99 (ms)	15.498	13.232
Mann-Whitney $U = 4500.0$, $p = 0.22$ (two-tailed); inter-quartile ranges overlap; both paths <50 ms at P99.		

The 0.65 ms median difference between the Fabric and PostgreSQL-only paths (Table 9) is statistically insignificant, as shown above. The absolute difference of 0.65 ms is well within baseline HTTP round-trip noise and meets the RQ2 read-overhead threshold of 50 ms with substantial headroom. The write-latency SLO is met in absolute terms (2,084 ms P50 vs. the 5 s threshold); the incremental overhead of the Fabric path over a PostgreSQL-only write was not separately measured, as no such bypass path exists in the framework design. Experiment 1 confirms the latency is orderer-dominated rather than database-dominated (PostgreSQL operating at ≈25% of its write-capacity reference, CPU below 23%), so the SLO headroom is structurally grounded, as summarized in Fig. 20.

6.3. Experiment 3: File-Size Independence of Ledger Latency

Because the ledger stores only a fixed-length SHA-256 hash and IPFS CID regardless of document size, Fabric commit latency should be invariant with respect to payload size. This experiment confirms that property across a 1–50 MB range. IPFS upload time was measured directly via the Kubo HTTP API (`curl` multipart, `pin=false`), isolating the IPFS-node upload cost from the secondary-pinning overhead present in the full REST path (≈234 ms on M1). Ten samples per file size were collected; total *server-side* latency is Fabric constant plus IPFS upload. This excludes client browser encryption time and network transmission from the client to the REST gateway, which are workload- and bandwidth-dependent and not modelled in this experiment.

Fabric commit latency was $2{,}132 \pm 20$ ms across all file sizes (1–50 MB), flat within run-to-run Raft variance. IPFS upload grew from 10 ms at 1 MB to 95 ms at 50 MB, consistent with near-linear Kubo chunk hashing. Total server-side P50 ranged from 2,142 ms to 2,226 ms at 50 MB, an ≈85 ms increase over a 50× size range, entirely attributable to IPFS. All points remain below the 5 s RQ2 threshold by more than 2×, confirming that even large legal evidence files register without exceeding acceptable latency for a non-interactive background operation (Fig. 21).

6.4. Experiment 4: Audit Verification Efficiency

This experiment quantifies operational audit-query latency for PostgreSQL and CSV-based verification over 1,000 events, and compares those results with the Fabric `GetHistoryForKey` measurement from Experiment 7 at a 107-entry document history. These paths provide different trust guarantees and are not identical workloads: PostgreSQL is the low-latency operational

index, CSV+SHA-256 is a manual verification baseline, and Fabric provides ledger-verifiable history. A PostgreSQL `audit_log` table was seeded with 1,000 real Fabric-anchored events. Automated query time was measured via `GET /api/audit?size=1000` over five runs. Manual verification time was the wall-clock cost of a Python script that fetches the CSV export and recomputes the SHA-256 digest of each of the 1,000 exported records and cross-checks them against the on-chain validation anchors: a computational lower bound, since operational manual audit additionally requires human download, inspection, and cross-referencing.

The automated PostgreSQL query completed in 3.9 ms median (mean 3.45 ms; n=10 trials, conventional median as the average of the 5th and 6th sorted values). Manual script verification took 86.3 ms, 22× slower in raw compute, and orders of magnitude slower in practice. The raw speedup, however, is not the point. The architectural contribution is the *complementary independence* of the two stores. PostgreSQL provides sub-5 ms operational dashboards with application/database-trigger-level append-only protection: row-level triggers block UPDATE and DELETE; a statement-level trigger blocks TRUNCATE (which bypasses row-level triggers in PostgreSQL); and TRUNCATE privilege is revoked from all application roles (Listing 6). The Fabric ledger provides cryptographic non-repudiation that PostgreSQL cannot replicate: any consortium peer can independently verify the complete audit trail via `GetHistoryForKey` without trusting the application server or its administrator [10, 12]. Compromise of one store does not affect the other. A malicious DB administrator who truncates the PostgreSQL log leaves the on-chain Fabric record intact and independently verifiable, as compared in Fig. 22.

6.5. Experiment 5: Resilience Under WAN Latency

WAN conditions were simulated in two configurations using Linux `tc netem`. The first configuration injected symmetric delay on the Docker bridge interface (`br-29a499b93a83`), affecting host-to-container HTTP connections at 0, 50, 100, and 150 ms RTT. The second configuration additionally applied `tc netem` delay to the `veth` interface of each orderer container, so that inter-orderer Raft traffic also experienced the injected latency; this configuration was measured at all four RTT levels. Throughput was measured at 200 concurrent clients (five 60-second trials per RTT level); commit latency was measured via 20 CLI samples of `RegisterDocument -waitForEvent` per level.

Table 10: Experiment 5 WAN Resilience: bridge-only delay vs. bridge plus per-orderer-container `veth` delay (200 clients, 5 trials per RTT level).

RTT	**Mean TPS (200 clients)**		**P50 Commit (ms)**	
	Bridge	Bridge + veth	Bridge	Bridge + veth
0 ms	68.2	69.5	2,140	2,141
50 ms	64.9	50.9	2,240	2,919
100 ms	62.3	47.0	2,357	3,132
150 ms	60.8	38.3	2,457	3,588

Bridge-only results (0–150 ms RTT). Throughput degraded from 68.2 TPS at 0 ms to 60.8 TPS at 150 ms RTT, an 11.0 % reduction. It remained at least 3.6× above the 16.7 TPS legal synthetic steady-state baseline at every tested level.

Commit latency grew from 2,140 ms to 2,457 ms (+317 ms at 150 ms RTT), consistent with ≈2 HTTP round-trips through the injected delay. The intermediate points (+100 ms at 50 ms RTT and +217 ms at 100 ms RTT) confirm predictable growth. This configuration isolates HTTP-layer gateway queuing because the injected delay does not reach container-to-container Fabric peer traffic (Fig. 23).

Bridge plus per-orderer-container results (0–150 ms RTT). With delay also applied to each orderer container's `veth` interface, the 0 ms baseline was 69.5 TPS and 2,141 ms P50 commit latency. These values match the Fabric commit constant of $2{,}132 \pm 20$ ms from Experiment 3.

Throughput declined to 50.9 TPS at 50 ms RTT, 47.0 TPS at 100 ms RTT, and 38.3 TPS at 150 ms RTT, a 44.8 % total reduction from baseline. P50 commit latency grew to 2,919 ms (+778 ms), 3,132 ms (+991 ms), and 3,588 ms (+1,447 ms), respectively. The larger degradation relative to bridge-only delay reflects Raft AppendEntries round-trip latency, now captured by the per-container delay.

At the highest-RTT, most-delayed point, some requests exceeded the 10 s client timeout; reported TPS is therefore successful-commit throughput. Even there, 38.3 TPS remains 2.3× above the 16.7 TPS baseline, and 3,588 ms remains 1.4 s below the 5 s write budget. The threshold is not approached anywhere in the tested range [12, 76]. Table 10 summarizes both configurations.

6.6. Experiment 6: Cryptographic Primitive Benchmark

All cryptographic operations in the framework are intended to execute client-side in the browser. Browser automation was unavailable in the measurement environment, so this experiment uses Node.js WebCrypto as a same-API fallback rather than claiming browser-performance results. The benchmark was executed on Linux with Node.js WebCrypto and produced 170 CSV rows: 17 measured operation/configuration entries with 10 trials each. Fig. 24 reports the operations most relevant to the paper's document workflow. The goal is to verify that password-based key derivation, document encryption, hashing, signing, verification, and key wrapping are not the dominant costs in the measured system.

PBKDF2-SHA256 at 600,000 iterations completed with a P50 latency of 100.44 ms over 10 trials, within the 1,000 ms interaction budget commonly used for preserving user flow and consistent with the work-factor guidance used in the framework [17, 53]. For 50 MB inputs, AES-256-GCM encryption measured 44.42 ms P50, AES-256-GCM decryption measured 61.94 ms P50, and SHA-256 hashing measured 44.43 ms P50. These values confirm that cryptographic processing is small relative to the Fabric write latency dominated by the 2 s `BatchTimeout`.

ECDSA P-256 document signing and verification were sub-millisecond in the same Node WebCrypto benchmark: signing measured 0.183 ms P50 and verification measured 0.097 ms P50. ECIES-style P-256 key wrapping measured 0.403 ms P50 and unwrapping measured 0.603 ms P50. ECIES is used primarily for compact per-recipient token storage rather than for a speed advantage: both ECIES and RSA wrapping costs are sub-millisecond in the Node WebCrypto benchmark. The ECIES-style P-256 wrapped-key token occupies 125 bytes, compared with 256 bytes for RSA-OAEP 2048, a 51.2 % reduction that directly reduces ledger and network storage per access-grant event.

6.7. Experiment 7: Blockchain Audit Trail Query Depth

Experiment 4 showed that PostgreSQL serves operational dashboards at 3.9 ms P50. The Fabric ledger's `GetHistoryForKey` path is slower, but it is the only measured path that provides cryptographic non-repudiation verifiable by any consortium peer. This experiment therefore measures the cost of ledger-verifiable history queries at a realistic workflow depth.

A benchmark document was seeded with 107 committed history entries: one `RegisterDocument` transaction followed by 106 `GrantAccess` invocations,

each committed in a separate block. This depth represents an actively shared document with periodic access-grant renewals across multiple firms. Ten trials were executed with `peer chaincode query` (CLI), measuring the full round trip from peer request, to CouchDB history scan, to response.

`GetHistoryForKey` completed in 135.5 ms P50 (mean 134.0 ms, range 123–145 ms, $\sigma = 6.1$ ms). The coefficient of variation is $<5\,\%$, indicating stable performance with no observed index-scan anomalies. Because the audit-history measurement was obtained through a peer chaincode query CLI path, whereas the main latency and throughput experiments use the REST-gateway workload path, these measurements are reported as distinct evaluation paths and should not be interpreted as identical workloads. Only the 107-entry depth was directly measured.

CouchDB evaluates `GetHistoryForKey` with a full sequential scan and no secondary index, so latency is expected to grow approximately linearly with history depth. For that reason, the 1,000-entry value in Fig. 25 is a back-of-envelope linear projection from the single measured depth, not an independently measured point. Extrapolating from 135.5 ms at 107 entries gives ≈1,266 ms at 1,000 entries. That value is suitable for background forensic reports, but it exceeds the 200 ms interactive threshold, which is crossed at approximately 158 entries. Multi-depth validation across deeper histories is left as future work.

For documents with deeper histories, operational queries should use the PostgreSQL `audit_log` path measured in Experiment 4 (3.9 ms P50). The Fabric path is reserved for contexts requiring cryptographic non-repudiation that no application-layer log can provide, and is the core answer to RQ1 [4, 71].

6.8. Experiment 8: BatchTimeout Sensitivity

Experiment 1 established that throughput at the default `BatchTimeout=2s` is gated by the Raft batch window rather than by concurrency. This experiment characterizes the throughput–latency trade-off as `BatchTimeout` is reduced, and tests whether the load generator, not the orderer, becomes the limiting factor at higher rates. At 50 concurrent clients, the network was rebuilt at each of 2 s, 500 ms, and 250 ms (`MaxMessageCount` fixed at 500), and ten 60-second sustained-throughput trials were taken per setting using the closed-loop load generator. The load generator's own CPU utilization was recorded each run to detect client-side saturation.

Sustained throughput was 68–70 TPS at 2 s, rose to 193.0 TPS at 500 ms, and then fell to 180 TPS at 250 ms. The 500 ms setting is therefore the

sustained-throughput sweet spot, providing an 11.6× margin over the 16.7 TPS synthetic steady-state baseline without architectural change.

The regression at 250 ms is reproducible across all ten trials and is *not* a load-generator artifact. Client CPU remained far from saturation: approximately 2 %, 11 %, and 14 % for the 2 s, 500 ms, and 250 ms settings, respectively. The likely cause is internal to the orderer: a 250 ms batch window cuts roughly twice as many small blocks as a 500 ms window, increasing per-block commit and validation overhead. The 250 ms setting also shows higher variance (137–206 TPS across trials) than 500 ms (174–217 TPS), slightly lowering net sustained throughput.

This resolves an apparent tool-dependent discrepancy. A fixed-count harness ranks 250 ms above 500 ms because it favors per-request latency and batch completion time. The sustained closed-loop measurement reported here is the representative throughput metric and identifies 500 ms as the operational optimum shown in Fig. 26.

6.9. Experiment 9: Fail-Closed Security Under Fabric Peer Outage

The threat model (Section 4.1) specifies a strict *fail-closed* architecture for protected document release during network partitions: if the Fabric peers become unreachable, the evaluated download path denies access rather than falling back to a database ACL. This experiment validates that behavior empirically. Under a steady 50-client ciphertext-download load, all three Fabric peers were stopped at $t = 30$ s and restarted at $t = 75$ s, a 45-second total Fabric peer outage. Fig. 27 shows the per-second timeline. Successful downloads dropped to exactly zero requests per second for the entire outage window; peers were restarted at $t = 75$ s, with normal operation resuming automatically after an approximately 15-second gRPC reconnection lag. During the outage, the service sustained 25–45 HTTP 503 fail-closed denials per second, securely blocking all tested download requests (1,340 denial responses and 2,013 PostgreSQL audit rows; the difference reflects additional system-level audit events recorded per denial beyond the HTTP response itself). `FABRIC_OUTAGE_ACCESS_DENIED` audit rows were written to the operational PostgreSQL log capturing the gRPC failure reason. On peer restart, normal Fabric evaluation resumed cleanly with no manual intervention.

This result substantiates the fail-closed behavior for the evaluated document-download path: a peer outage degrades to a secure denial state, prioritizing confidentiality over availability. A malicious DB administrator who deliberately induces an outage cannot force a fallback to the database ACL to

bypass chaincode authorization on this path.

7. Limitations and Future Work

7.1. Limitations

Each limitation is accompanied by a concrete mitigation path.

- **Coarse-grained intra-organization access control.** The prototype `CheckAccess` chaincode (Listing 1, lines 37–39) grants read access to any peer whose MSP organization matches the document owner's organization (`doc.OwnerOrg == userOrg`), without requiring an individual ACL entry. This constitutes an owner-org bypass: any member of `LawFirmA` can access any document registered by any user in `LawFirmA`. The framework design specifies per-user grants for all principals; closing this gap in production requires removing the organization-level fallback and enforcing explicit grants for intra-organization peers, consistent with a zero-trust access model.

- **Access-decision replay granularity.** The `CheckAccess` chaincode uses the Fabric transaction timestamp (`ctx.GetStub().GetTxTimestamp()`) for grant-expiry evaluation, ensuring all endorsing peers assess the same proposal-embedded timestamp regardless of local clock offset. Exact historical replay of individual access decisions would additionally require committing each access-decision event on-chain with its evaluated timestamp and result; this is identified as a production hardening item and deferred to future work. Expired-grant cleanup (as opposed to evaluation) follows a separate eventual-consistency model via the asynchronous `CleanExpiredTokens` job (Section 5.4).

7.1.1. Geographic Distribution and WAN Latency

Experiment 5 evaluated WAN conditions up to 150 ms RTT under two delay configurations: bridge-only delay at the HTTP layer, and bridge plus per-orderer-container `veth` delay, which also captures inter-orderer Raft AppendEntries latency.

Under bridge-only delay, throughput degraded by 11.0 % (68.2 to 60.8 TPS at 200 clients), and commit latency grew from 2,140 ms to 2,457 ms. When inter-orderer Raft traffic was also delayed, throughput degraded by 44.8 % (69.5 to 38.3 TPS), and commit latency grew from 2,141 ms to 3,588 ms. In

both configurations, the 16.7 TPS synthetic steady-state baseline was cleared by at least 2.3×, and commit latency remained below the 5 s operational threshold.

Performance in globally distributed, multi-continent deployments has not been validated at production scale. Mitigations include reducing `BatchTimeout` (500 ms reached 193.0 TPS and 250 ms reached 180 TPS in Experiment 8, both upper bounds for the evaluated three-organization deployment) or placing orderer nodes with regional awareness. Hyperledger Fabric's modular ordering service supports such channel and placement choices [12, 73].

The WAN experiment also does not independently test orderer-node failover, multi-peer failure combinations, or recovery during concurrent production-like legal workloads. Likewise, the 16.7,TPS workload model is a synthetic capacity-planning assumption rather than a replay of real legal transaction logs. Future empirical work should replay anonymized legal document-management transaction logs and test peer failure, orderer failover, and recovery under geographically distributed load.

7.1.2. Crash-Fault Tolerance vs. Byzantine Fault Tolerance

The 3-node EtcdRaft ordering service provides CFT: it tolerates the failure of any single ordering node. CFT does not protect against Byzantine orderer behavior. In the proposed legal consortium, this risk is mitigated architecturally: all ordering nodes are operated by legally bound institutions under audited consortium agreements, and Fabric's execute-order-validate pipeline ensures peers independently validate every block regardless of orderer behavior [10]. For deployments requiring a stricter Byzantine threat model, Hyperledger Fabric v3.0 introduces SmartBFT ordering as a production-hardening path that does not change the document-management chaincode semantics, although it does require ordering-service and channel-configuration changes [43, 44, 61].

7.1.3. Key Management and Browser Dependency

Three key management limitations arise. First, key rotation is owner-initiated: re-encryption of all affected document keys after a compromise requires the document owner's browser session. An interim production-compatible direction is proxy re-encryption (PRE), where an owner authorizes a proxy to transform a wrapped document key from one recipient key to another without revealing the document key to the proxy [77, 78, 6]. This

paper does not implement PRE; it is identified as a future key-rotation mechanism that would require a careful construction, threshold governance, and implementation-level security analysis. Second, private keys are stored in browser `localStorage` under PBKDF2-derived AES-256-GCM protection; key loss through browser data clearing is permanent without a backup procedure. Third, all cryptographic operations use the browser WebCrypto API [15], unavailable in headless or server-side contexts. The production resolution for both the second and third limitations is WebAuthn/FIDO2 or HSM-backed key protection, deployed in a manner consistent with SP 800-63-4 AAL2/AAL3 requirements [58, 59, 46, 57].

7.1.4. On-Chain Metadata Privacy

Document content is protected by AES-256-GCM and is never stored on the ledger. Metadata remains visible to channel members, including timestamps, invoking MSP identity, CIDs, and access-event sequences. A channel member can infer case activity patterns by observing transaction frequency and CID patterns over time, even without decrypting content.

Hyperledger Fabric PDCs restrict sensitive fields to authorized subsets of channel members [34, 79]. However, each collection hash is still written to the main ledger [64]. Possible mitigations include narrower channels or PDC scopes, metadata minimization, batching, cover traffic, private access protocols, and zero-knowledge proofs for selected policy checks [80].

The plaintext hash $\mathcal{H}_D$ is also stored in main channel state. Any peer holding a candidate document can therefore confirm its registration without access to the ciphertext or decryption key. This leak is intentional for the prototype's integrity-verification workflow, but production systems should move $\mathcal{H}_D$ into a Fabric Private Data Collection or replace it with a keyed commitment available only to authorized auditors.

7.1.5. Authentication Assurance Level

The prototype implements AAL1 under NIST SP 800-63-4 [58]: username/password with signed JSON Web Tokens [60]. AAL1 is adequate for the prototype evaluation of normal-path chaincode access checks and audit mechanisms, but it is insufficient for production legal consortium deployments. The authentication gateway is explicitly designed for AAL2 and AAL3 upgrade without chaincode modification.

7.1.6. Identity Database Public Key Integrity

The PostgreSQL identity database stores each user's public keys (`pk_ecies`, `pk_sign`). The audit architecture protects the `audit_log` table against modification, but the identity table is not subject to equivalent cryptographic protection. A database administrator with write access could substitute a malicious public key for a legitimate user's `pk_ecies`; subsequent `GrantAccess` invocations would wrap k_{enc} under the attacker's key and commit this subverted grant to the ledger as a valid transaction. The ledger would record a cryptographically correct but semantically compromised key exchange. The production mitigation is to anchor public key registrations to the Fabric ledger at enrollment time, for example by invoking a `RegisterIdentity` chaincode function that commits each user's public keys as a ledger asset, making any post-registration key substitution detectable by any consortium peer. This is deferred to future work alongside full per-user X.509 enrollment (Section 7.2).

7.1.7. IPFS Retrieval Timeout and Thread Management

The prototype does not enforce an explicit timeout on IPFS `GetFile` calls in the document download path. Under normal operation, the Fabric `CheckAccess` invocation completes before the IPFS retrieval begins. However, if the IPFS swarm experiences a node failure, disk fault, or network partition *after* chaincode authorization succeeds, the Spring Boot HTTP thread handling the request may block indefinitely waiting for the ciphertext. Under sustained load, this could exhaust the application server's thread pool, causing a cascading failure that degrades or disables endpoints unrelated to IPFS. The production mitigation is a circuit-breaker pattern with a hard timeout (e.g., 5 s) on all IPFS retrieval calls, returning HTTP 504 to the caller and logging an `IPFS_RETRIEVAL_TIMEOUT` audit event. This ensures IPFS unavailability degrades gracefully rather than propagating to a full service outage. Experiment 9 validates fail-closed behavior only for the document download path (`downloadCiphertext()`); the fail-closed behavior of the upload, grant, revoke, and audit-query paths is asserted by design (all paths invoke the same `FabricGatewayService` pattern and return `HTTP 503` on any `ServiceUnavailableException`) but was not independently validated empirically and is deferred to future work.

7.1.8. GDPR and Regulatory Compliance

The framework presents a structural tension with the European General Data Protection Regulation (GDPR) [81]. Article 17 grants data subjects a

right to erasure, requiring personal data to be deleted upon request. Hyperledger Fabric's append-only ledger provides tamper-evidence precisely because committed transactions cannot be deleted.

The proposed framework partially resolves this tension through design decomposition. Document content is stored off-chain in IPFS and can be unpinned and garbage-collected, effectively erasing the ciphertext from distributed storage [36, 37]. The on-chain ledger retains SHA-256 hashes, CIDs, timestamps, and MSP identity references. Whether these values constitute personal data under GDPR Article 4(1) [81] is context-dependent and legally contested. Although hashes and CIDs cannot reconstruct the original document without the ciphertext and key, data-protection analysis often treats cryptographic hashes of personal data as *pseudonymised* rather than anonymous, especially when linked to persistent institutional or user identities [65, 64].

Article 17 compliance is therefore not fully solved by the prototype. Off-chain erasure can remove encrypted document payloads from IPFS, but append-only on-chain metadata may remain linkable to a person, case, or institution depending on the deployment context. Production use in jurisdictions applying GDPR erasure obligations requires a deployment-specific legal basis and governance process, together with technical hardening such as metadata minimization, narrower channels, private data collections, off-chain redaction workflows, or redactable-ledger mechanisms where legally required. Cross-jurisdiction data-residency obligations must also be resolved at deployment time because the relevant legal exposure depends on where IPFS pinning nodes, Fabric peers, and orderers are operated, and on which jurisdictions treat on-chain hashes, CIDs, timestamps, or MSP identity references as personal or regulated data.

Accordingly, the framework treats the ledger as evidentiary rather than dispositive: the consortium agreement, not the chaincode, remains the legal instrument governing access and erasure obligations. If on-chain metadata is treated as personal data in a deployment jurisdiction, full compliance may require a legal determination specific to the consortium or a technical mechanism for selective record modification under governance. Redactable-blockchain mechanisms based on chameleon hashes are a possible research direction, but adopting them in Fabric would require modifying the block-hash construction and defining threshold governance for trapdoor use [77, 82]. Without such advanced cryptographic deletion methodologies or explicit off-chain zero-knowledge verification patterns, the system may face significant compliance

barriers to deployment in strict European jurisdictions handling personally identifiable legal records, a regulatory risk assessment consistent with EDPB guidance on pseudonymization and blockchain data governance [83, 84].

7.1.9. Integration with Legacy Legal Systems

The prototype operates as a self-contained system. Production deployment in an active legal consortium requires connectors to existing practice management software, court filing platforms, and document repositories. The following outlines the four architectural concerns a production integration must address.

(a) API surface and capability exposure. The Spring Boot REST gateway exposes four capability groups relevant to legacy connectors: document registration (`POST /documents/upload`), document retrieval (`GET /documents/{id}/download`), access grant and revocation (`POST /documents/{id}/grant`, `POST /documents/{id}/revoke`), and audit trail query (`GET /api/audit`). A legacy system with read-only audit requirements needs only the last endpoint and does not interact with the Fabric layer directly [67].

(b) Identity boundary: OAuth2 tokens to Fabric X.509 identities. Legacy practice management systems typically issue OAuth2 bearer tokens for authenticated sessions. At the integration boundary, a token exchange service must map a validated OAuth2 identity to a Fabric X.509 certificate issued by the consortium Fabric CA. In the prototype this mapping is handled under the shared administrative MSP identity; in production each legacy user identity must be enrolled with the Fabric CA and issued a certificate so that chaincode invocations are attributable to a specific principal [45, 60].

(c) Boundary security controls. The integration boundary must enforce: TLS mutual authentication (mTLS) between the legacy connector and the Spring Boot gateway to prevent credential interception; rate limiting on write endpoints to prevent batch flooding of the Raft orderer beyond its `BatchTimeout` capacity; and payload inspection to reject uploads that exceed the IPFS node's configured maximum object size before a Fabric transaction is attempted [6].

(d) Phased migration path. A four-phase migration is recommended. Phase 1: read-only audit access, in which the legacy system queries `GET /api/audit` to surface blockchain-verified audit records alongside its existing log. Phase 2: read-write with fallback, in which document registration and retrieval route through the gateway but the legacy system retains its own ACL as a parallel authority. Phase 2 is a pre-enforcement migration phase for

non-Fabric-classified documents only; once a document is classified as Fabric-governed, its release path must not fall back to legacy ACL authority. During Phase 2, legacy ACLs operate in shadow or advisory mode alongside Fabric enforcement, not as an availability substitute. Phase 3: Fabric-governed ACL enforcement with fallback policy disabled, in which the `CheckAccess` chaincode becomes the access-control authority for protected document release and the legacy ACL is retired. Phase 4: full per-user X.509 enrollment, in which the shared administrative MSP identity is replaced by per-user certificates and hardware-backed signing keys. Each phase is independently deployable and requires no chaincode modification [63, 6].

7.2. Future Work

Three directions dominate the path to production. First, per-user Fabric CA enrollment with HSM-backed or WebAuthn/FIDO2 key storage [57, 45, 59] would replace the shared administrative MSP identity and the password-protected `localStorage` key store, closing the custodial-attribution gap identified in Section 5.1; the same infrastructure supports a consortium-governed key-recovery workflow with KMS/HSM-backed escrow, threshold administrative approval, and auditable re-wrapping of recovered keys, so that no single administrator gains unilateral access to user private keys. Second, formal verification of the `CheckAccess` and `RegisterDocument` chaincode functions would provide machine-checked guarantees of access-control completeness and absence of chaincode-level bypasses under the modeled policy [85]. Third, privacy-preserving access verification, including zero-knowledge proofs for selected policy checks and batching or cover-traffic mechanisms to reduce timing and frequency leakage, would address the on-chain metadata exposure discussed in Section 7 [80, 34].

Further directions include regulator-facing compliance interfaces (read-only audit dashboards and joint endorsement policies such as `AND('LawFirmAMSP.peer', 'RegulatorMSP.peer')`), real-time anomaly detection over the low-latency audit feed [86], decentralized identity integration bridged to Fabric MSP enrollment [87], cross-network evidence verification via relay mechanisms or verifiable credentials [88], cross-version CID chaining for document-version provenance, formal incentive mechanisms for private-swarm pinning, and a comparative complexity and cost model spanning application-only, append-only database, audit-log, and on-path authorization architectures.

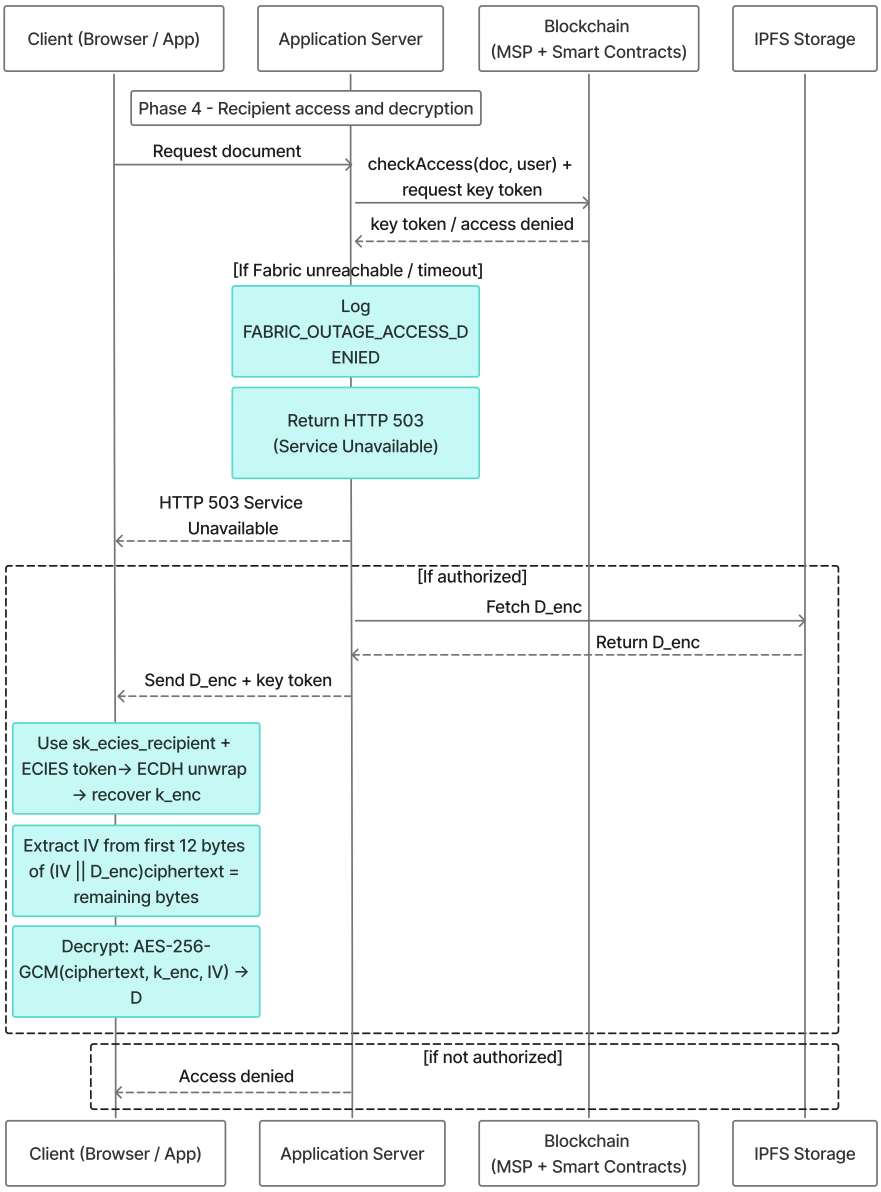


Figure 11: Phase 4: Recipient unwraps k_{enc} with sk_{ecies}, extracts IV from first 12 bytes of stored ciphertext, and decrypts locally. Server delivers only ciphertext and wrapped key token.

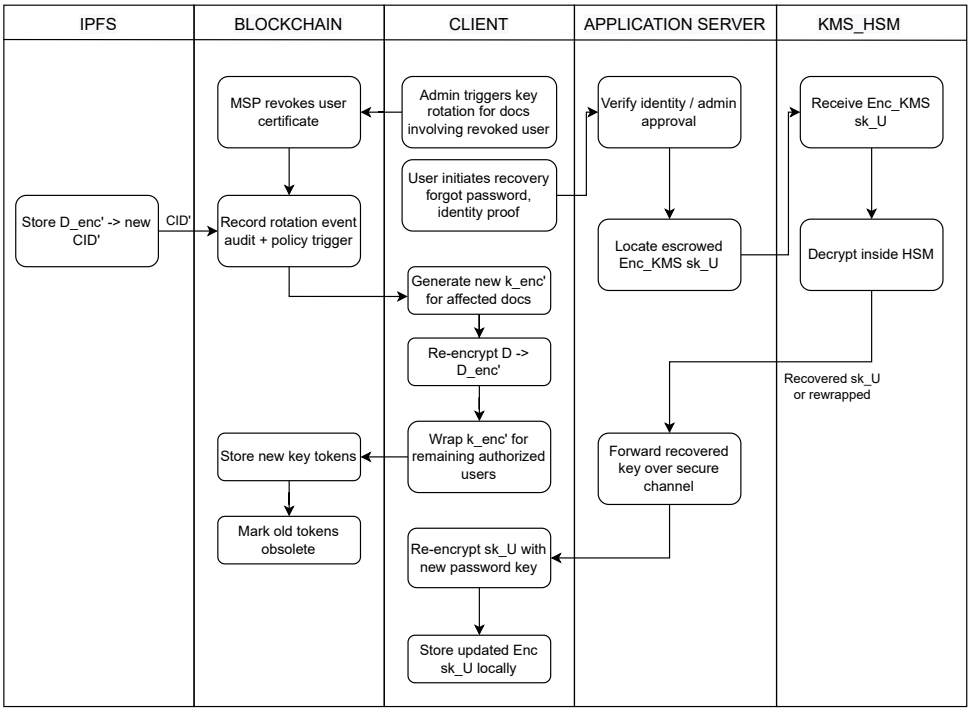


Figure 12: Phase 5: Key recovery via consortium KMS/HSM and revocation workflow (framework design, not implemented in prototype). Re-encryption requires the document owner's browser; the server does not hold k_{enc}.

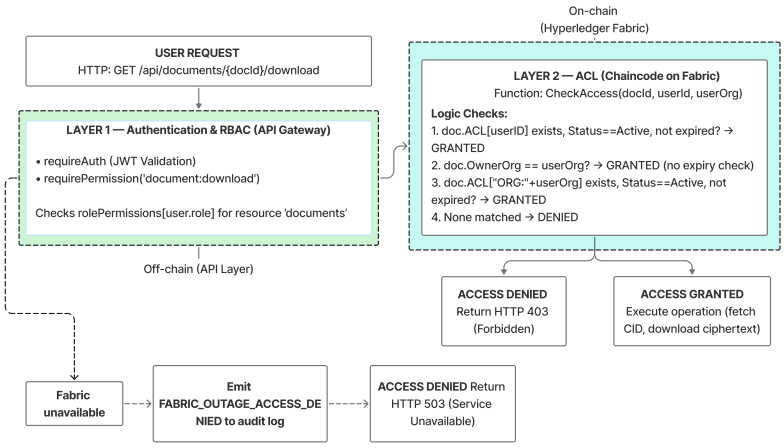


Figure 13: Two-layer access control pipeline: API-gateway JWT RBAC, on-chain `CheckAccess`, and fail-closed `FABRIC_OUTAGE_ACCESS_DENIED` blocking.

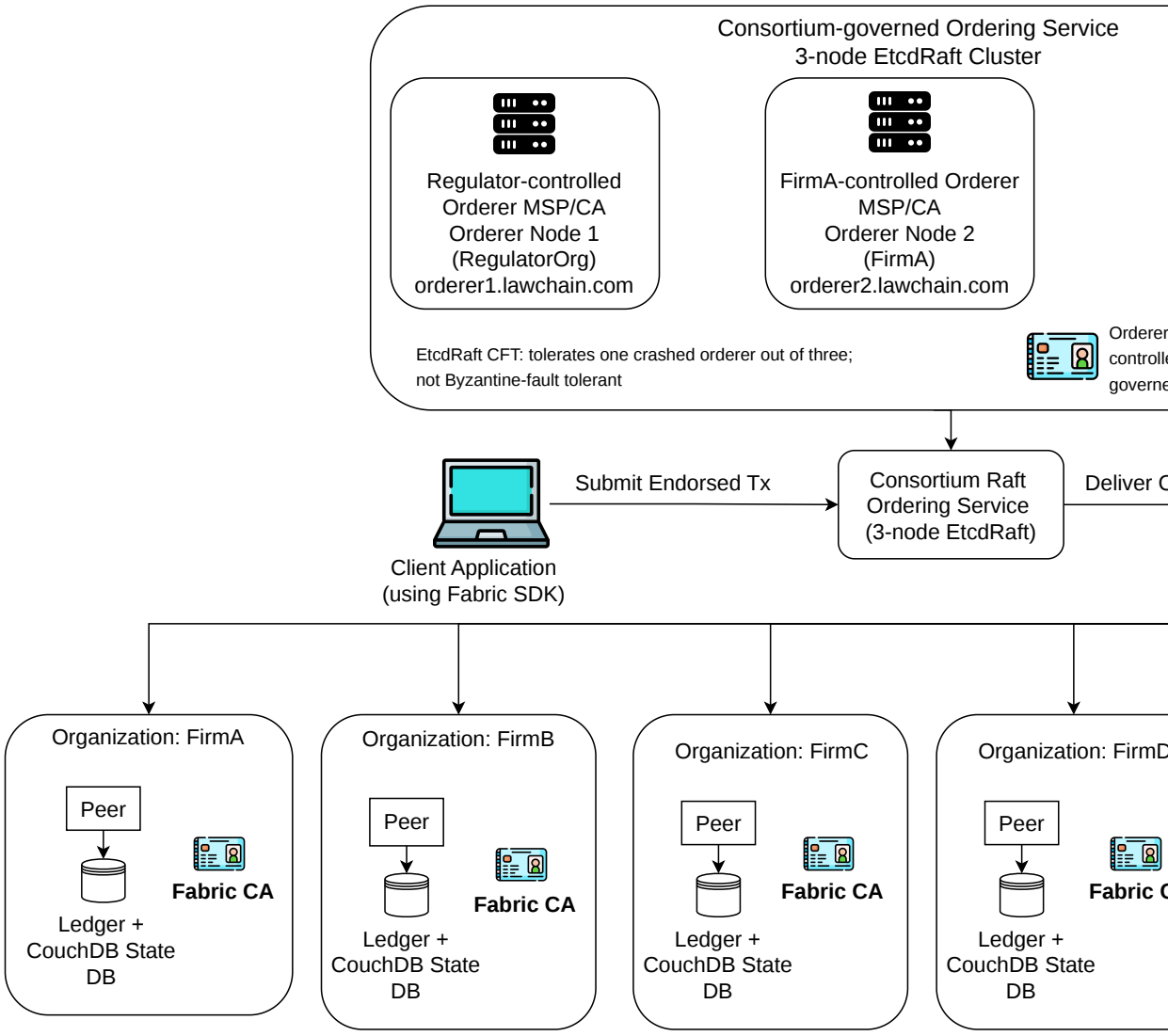


Figure 14: Six-peer-organization Fabric reference topology with a consortium-governed three-node EtcdRaft ordering service. The orderer nodes are operated by the regulator, FirmA, and FirmB under member-controlled orderer identity administration or multi-party CA governance. Raft provides crash-fault tolerance for one orderer failure; Byzantine-fault-tolerant ordering is outside the prototype scope.

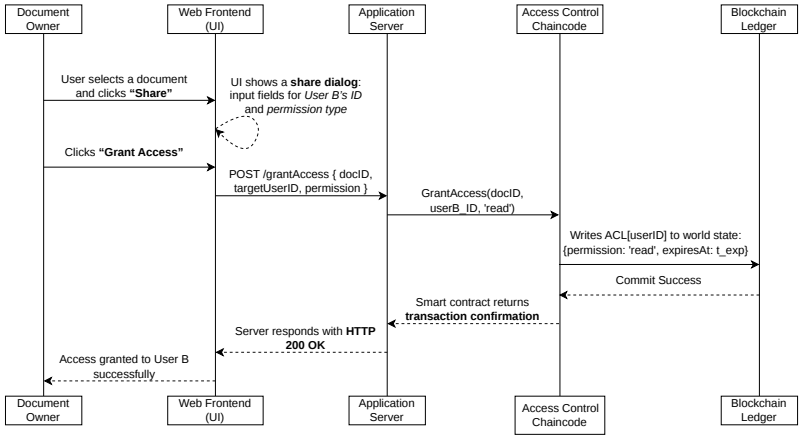


Figure 15: Sequence diagram for granting access permissions. The `GrantAccess` chaincode writes a time-bounded capability token to the on-chain ACL map, producing a ledger-verifiable record of the delegation.

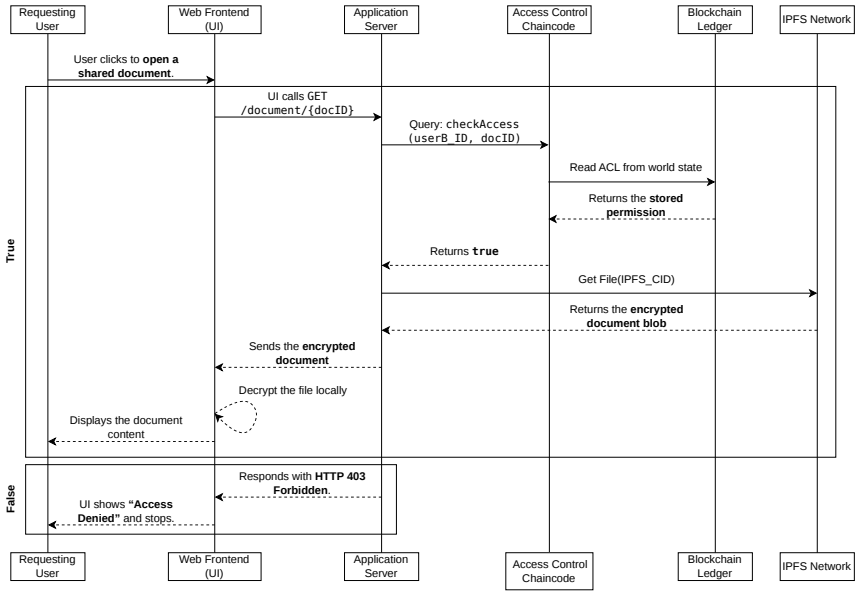


Figure 16: Access verification and document retrieval: Layer 1 (RBAC) and Layer 2 (`CheckAccess`) authorization, followed by ciphertext release on success or HTTP 403 denial on failure. Fail-closed handling of Fabric unavailability is shown separately in Fig. 27 and Listing 5.

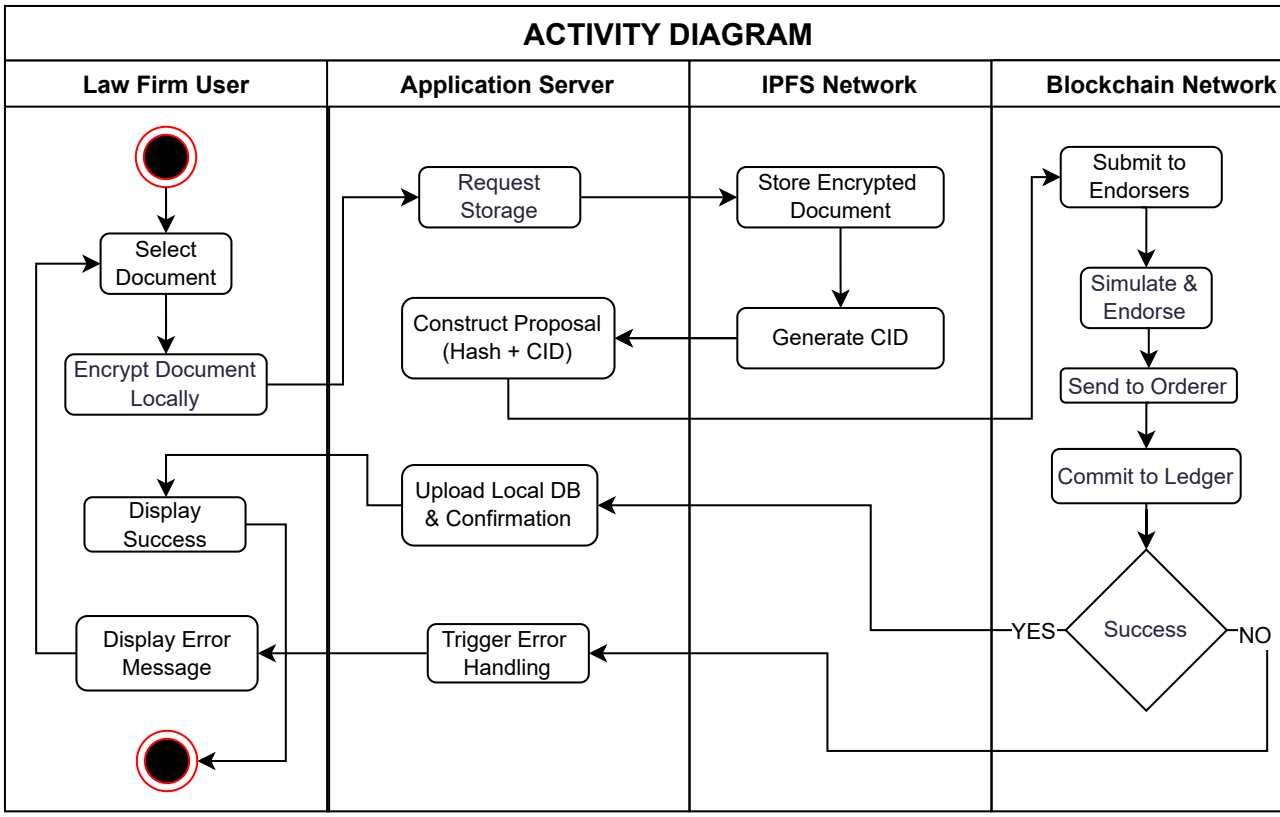
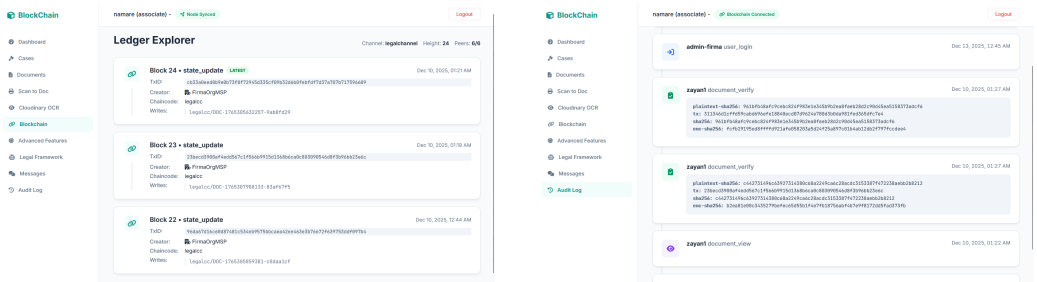


Figure 17: Secure document upload and registration activity diagram. Strict client-side encryption is enforced before IPFS storage and blockchain anchoring proceed. The measured prototype fully implements this confidentiality boundary.



(a) Ledger explorer (b) Audit log

Figure 18: Audit and transparency views of the prototype interface.

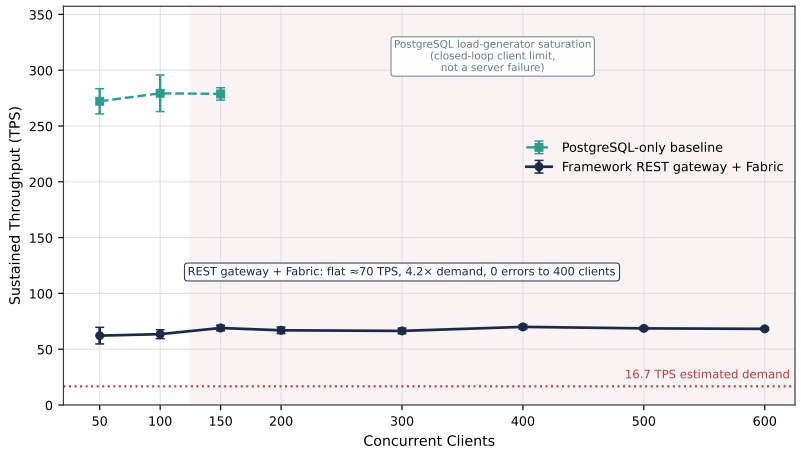


Figure 19: Experiment 1 scalability under concurrent load on Linux `x86_64`: Fabric REST-gateway throughput compared with the PostgreSQL-only baseline, the client-side saturation region, and the Experiment 6.8 `BatchTimeout` tuning reference.

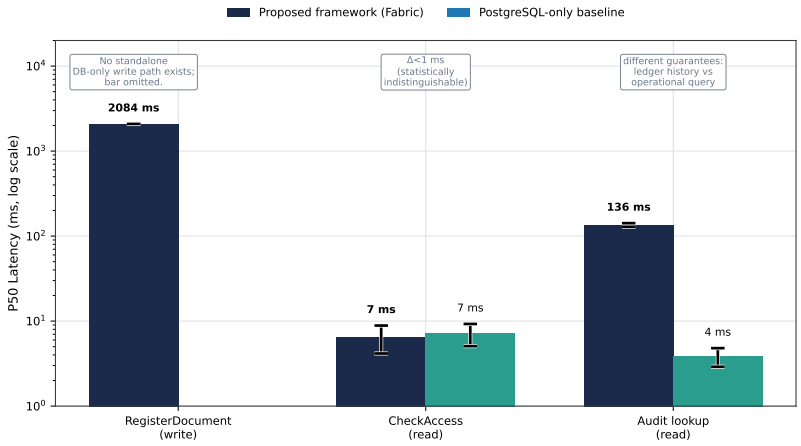


Figure 20: Experiment 2 function-level latency (P50, log scale; Linux, JVM-warmed, 100 samples): `CheckAccess`, `RegisterDocument`, and audit-query trust/latency baselines.

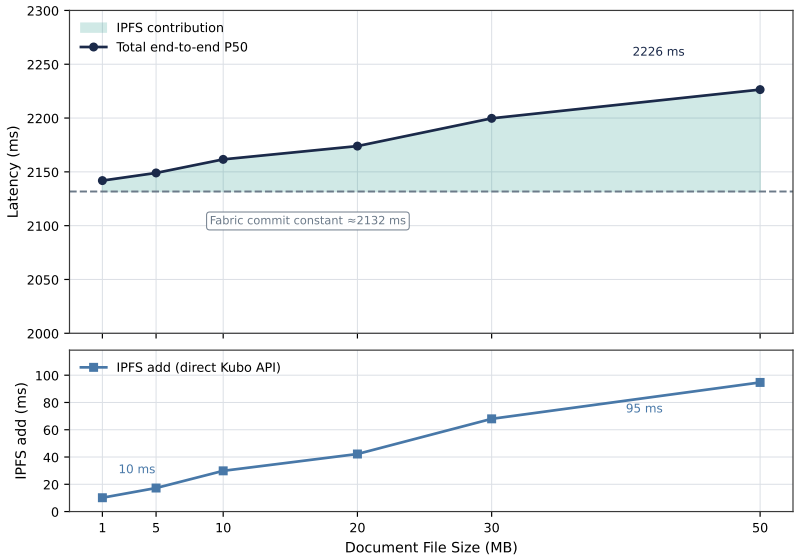


Figure 21: Experiment 3 file-size independence of ledger latency (Linux, direct Kubo API), separating server-side P50, IPFS upload range (10–95 ms), and constant Fabric commit latency.

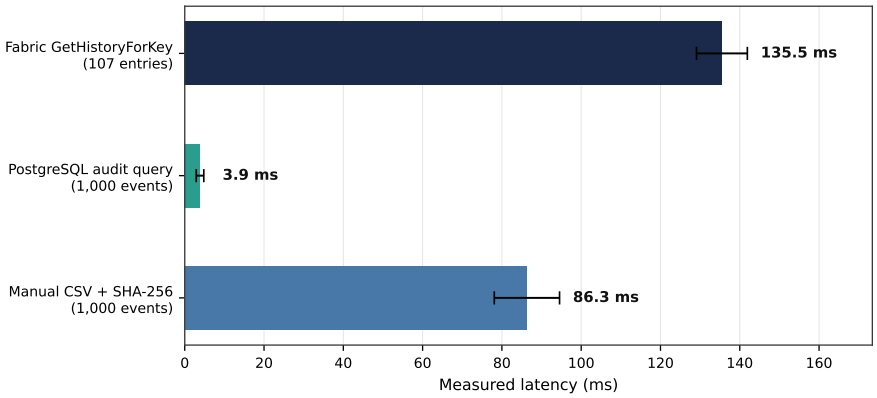


Figure 22: Experiment 4 audit verification efficiency across PostgreSQL queries, manual CSV + SHA-256 verification, and Fabric `GetHistoryForKey` ledger-verifiable history.

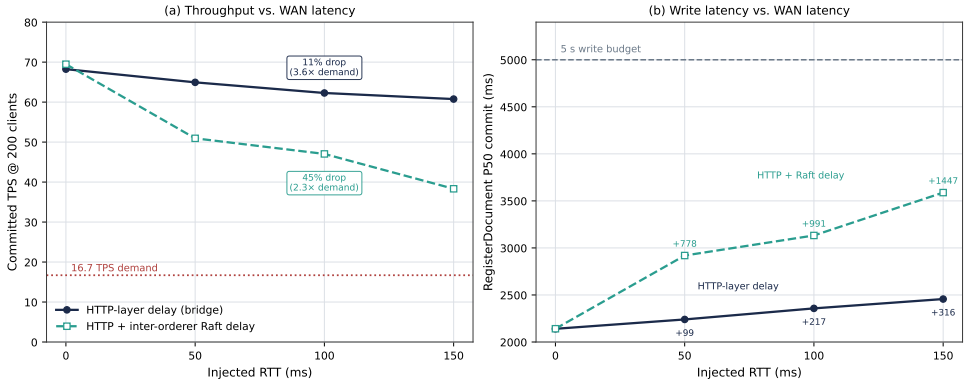


Figure 23: Experiment 5 WAN resilience (`tc netem`, Linux `x86_64`, 200 clients), comparing bridge-only and bridge + `veth` delay effects on TPS and `RegisterDocument` latency.

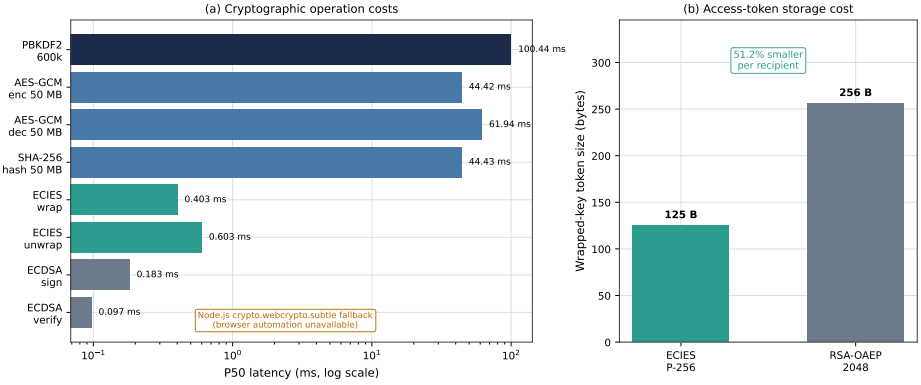


Figure 24: Experiment 6 cryptographic primitive benchmark using Node.js WebCrypto fallback (Linux): PBKDF2-SHA256, AES-GCM, SHA-256, ECDSA, ECIES, and RSA-OAEP token-size comparison.

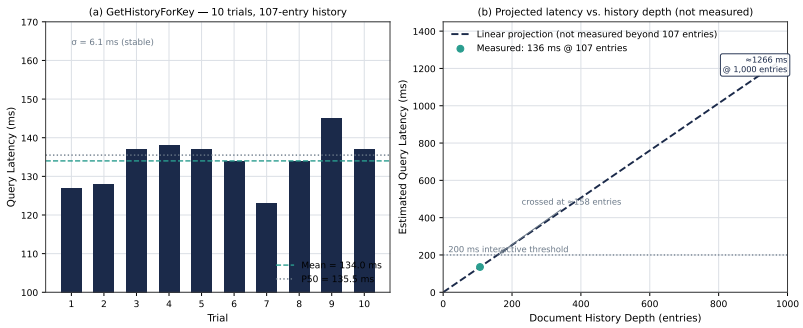


Figure 25: Experiment 7 Fabric `GetHistoryForKey` latency: 107-entry history depth measured directly; deeper data points are linear projections from this single measurement, not independently measured values. Multi-depth validation is future work.

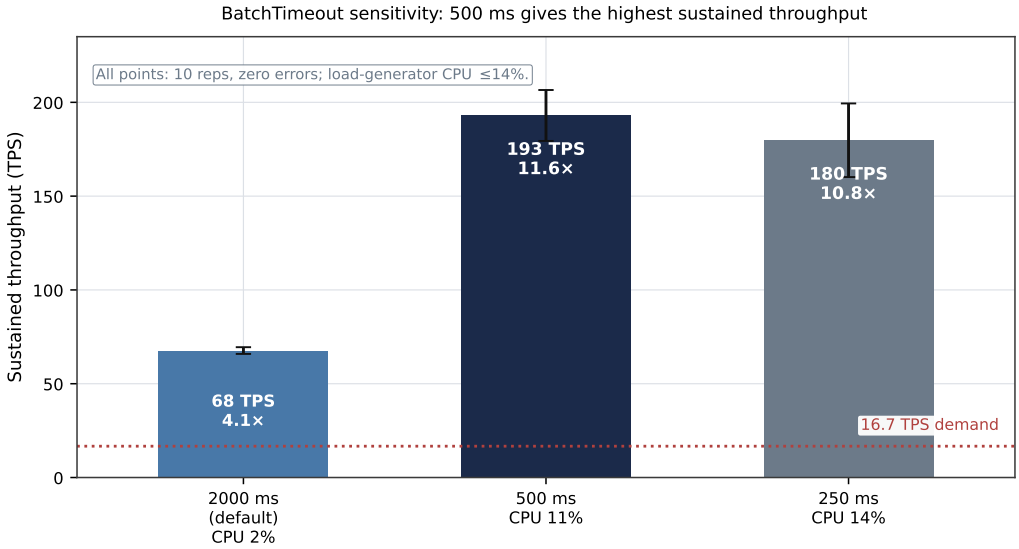


Figure 26: Experiment 8 `BatchTimeout` sensitivity (Linux `x86_64`, 50 clients), showing default 2 s, 500 ms, and 250 ms throughput with zero-error runs.

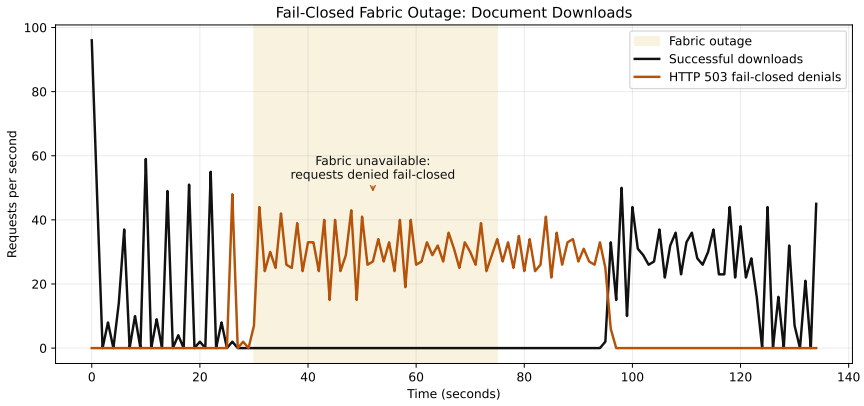


Figure 27: Experiment 9 fail-closed Fabric peer outage: zero successful downloads during the 45 s outage window; 1,340 HTTP 503 denial responses and 2,013 PostgreSQL audit rows (the difference reflects additional system-level audit events per denial); normal operation resumes after approximately 15 s gRPC reconnection lag.

8. Discussion

This section interprets the experimental results in relation to the three research questions and clarifies the boundaries of the measured prototype. The framework combines Hyperledger Fabric, encrypted IPFS storage, and a REST-based application gateway. Its central contribution is not a new access-control primitive, not a Byzantine-fault-tolerant ordering protocol, and not a production-ready legal deployment. Instead, the work evaluates the practical effect of moving the document-release authorization boundary from application-only trust to ledger-verifiable access-control state on the normal release path, with fail-closed behavior empirically confirmed for the download path. The prototype still has explicit boundaries, including server-managed Fabric MSP credentials and a two-node private IPFS swarm, but it implements strict client-side encryption and, on the evaluated document-download path, a fail-closed security policy during Fabric unavailability.

RQ1 asked whether a permissioned blockchain architecture can make legal-document access decisions and audit events independently verifiable while preserving confidentiality during Fabric unavailability. The answer is affirmative within the evaluated scope. Access-control *state*, namely grants, revocations, and document registrations, is independently verifiable by consortium peers. `CheckAccess` derives each download decision from that state at request time, although individual query results are not themselves committed as transactions.

This result has two important boundaries. First, ledger transactions remain attributable to the prototype's server-managed MSP identity until per-user Fabric CA enrollment is implemented (Table 7). Second, per-user `CheckAccess` enforcement was demonstrated for cross-organization access. Intra-organization access still falls back to organization-level membership; removing that fallback is the primary production hardening step.

The dual-store audit design separates operational speed from independent verifiability. PostgreSQL provides low-latency operational audit queries (3.9 ms P50 for 1,000 events, Experiment 4). Fabric `GetHistoryForKey` provides ledger-verifiable history (135.5 ms P50 at a 107-entry document history, Experiment 7). The 107-entry history is shallow relative to production legal lifecycles, so deeper-history scaling remains future work.

Under the stated threat model, a malicious database administrator cannot silently rewrite Fabric history. A compromised IPFS node also cannot undetectably modify document ciphertext because $\mathcal{H}_D$ is anchored on-chain

at registration. During Fabric peer outages, the evaluated download path denied all downloads and logged `FABRIC_OUTAGE_ACCESS_DENIED` events to the operational PostgreSQL audit log. Anchoring those outage denials on Fabric would require queue-and-replay support.

RQ2 asked whether chaincode-based access evaluation adds acceptable overhead for interactive legal workflows. The read-path threshold is met. Fabric `CheckAccess` and the PostgreSQL-only ACL baseline are statistically indistinguishable (6.51 ms vs. 7.16 ms P50, Mann-Whitney U=4500.0, p=0.22, n=100), and both remain below 15.5 ms at P99.

The write-latency SLO is also met in absolute terms: `RegisterDocument` measured 2,084 ms P50 against the 5 s threshold. The incremental overhead over a PostgreSQL-only write was not measured because the framework has no database-only registration path. Experiment 1 indicates that latency is orderer-dominated rather than database-dominated, with PostgreSQL at ≈25% of its write-capacity reference and CPU below 23%.

File-size experiments show that Fabric commit latency is independent of payload size because the ledger stores only metadata, hashes, and CIDs. WAN experiments further show that, even with 150 ms RTT applied to inter-orderer Raft traffic, throughput remains above the estimated workload and write latency remains below 5 s. The file-size experiments report server-side P50; client-side AES-256-GCM encryption adds approximately 44 ms P50 for a 50 MB input, as measured in Experiment 6.

RQ3 asked whether the framework sustains throughput sufficient for realistic legal workload peaks. The measured REST-gateway Fabric deployment sustains approximately 68–70 TPS across 50–600 concurrent clients, about 4.2× above the estimated 16.7 TPS workload for a 1,000-user organization issuing one document action per minute per user. Reducing `BatchTimeout` to 500 ms produced the highest measured throughput, 193.0 TPS, under the evaluated three-organization deployment, an 11.6× margin without changing the application architecture.

Experiment 9 confirms strict fail-closed enforcement on the document-download path. Successful downloads fell to exactly zero for the full 45-second Fabric unavailability window, while HTTP 503 denials continued at 25–45 per second. Every denied attempt was recorded in the operational PostgreSQL audit log, consistent with the outage-anchoring limitation noted under RQ1.

Normal operation resumed automatically after an approximately 15-second gRPC reconnection lag. The upload, grant, revoke, and audit-query paths enforce the same policy by design through the same `FabricGatewayService`

pattern, but were not independently validated in the experiment campaign and are identified as future empirical work.

The evaluation also shows that the cryptographic layer is not the dominant performance cost. Using Node.js WebCrypto as a same-API fallback, PBKDF2-SHA256 at 600,000 iterations measured 100.44 ms P50, AES-GCM encryption of a 50 MB document measured 44.42 ms P50, and AES-GCM decryption measured 61.94 ms P50. ECDSA signing and verification, and ECIES wrapping and unwrapping, were sub-millisecond in the measured environment. ECIES-style P-256 is used primarily for compact per-recipient key distribution: its 125-byte wrapped-key token is 51.2 % smaller than the 256-byte RSA-OAEP 2048 token.

9. Conclusion

This paper presented a hybrid framework for legal document management that combines Hyperledger Fabric, encrypted IPFS storage, and a REST-based application gateway. The core contribution is an empirical systems baseline for moving legal-document release decisions from application-only authorization to Fabric-backed `CheckAccess` evaluation on the normal download path. The work does not claim to introduce a new access-control primitive, a Byzantine-fault-tolerant ordering protocol, or a production-ready legal deployment.

The results show that this design can provide ledger-verifiable access-control state while maintaining practical performance for interactive legal workflows. Fabric access checks were statistically indistinguishable from the PostgreSQL-only ACL baseline on the evaluated read path, and the REST-gateway Fabric deployment sustained throughput above the estimated workload baseline. The fail-closed outage experiment further showed that successful downloads fell to zero during Fabric unavailability, confirming the evaluated download path's strict denial behavior.

These results substantiate the contributions set out in Section 1.3: a measured relocation of the enforcement boundary to ledger-verifiable access-control state, chaincode-enforced time-bounded grants and revocation, a dual-store audit design separating low-latency operational queries from ledger-verifiable history, and a nine-experiment evaluation spanning throughput to fail-closed outage behavior.

Table 7 documents the gaps between framework design and current implementation, bounding the measured prototype relative to the framework design.

To the best of the authors' knowledge, none of the prior Fabric + IPFS legal-document systems in Table 1 report the latency cost of placing `CheckAccess` on the critical download path. The negligible read overhead measured here (6.51 ms vs. 7.16 ms P50, a 0.65 ms difference, n=100) confirms that this architectural choice imposes negligible interactive overhead. It also establishes a reproducible baseline against which future on-path enforcement designs can be evaluated.

Future work will focus on per-user Fabric CA enrollment, HSM/WebAuthn-backed key protection, privacy-preserving access verification, formal verification of chaincode policy logic, stronger IPFS persistence governance, and compliance mechanisms for jurisdictions where append-only on-chain metadata raises erasure or data-protection concerns.

Appendix A. Implementation listings

This appendix collects the implementation listings referenced in the main text: browser-side encryption and key derivation, server-side signature verification, the fail-closed download path, and the audit-log trigger schema.

Listing 2: Browser AES-256-GCM document encryption. Fresh 256-bit key and 96-bit IV generated per document via `window.crypto.getRandomValues`. SHA-256 hash computed of plaintext *before* encryption. Server receives only IV-prepended ciphertext.

```
// frontend/src/lib/crypto.ts
export async function encryptDocument(
    file: ArrayBuffer
): Promise<EncryptedDocument> {
  const key = await subtle.generateKey(
      { name: 'AES-GCM', length: 256 },
      true, ['encrypt','decrypt'])
  const iv = crypto.getRandomValues(new Uint8Array(12))
  const [ciphertext, plainHashBuffer, rawKey] = await Promise.all([
    subtle.encrypt({ name: 'AES-GCM', iv }, key, file),
    subtle.digest('SHA-256', file),  // Plaintext hash
    subtle.exportKey('raw', key),
  ])
  // Hash the resulting ciphertext to bind the IPFS payload
  const ivAndCiphertext = concat(iv, ciphertext)
  const cipherHashBuffer = await subtle.digest('SHA-256', ivAndCiphertext)
  return {
    ciphertextB64: bytesToBase64(new Uint8Array(ciphertext)),
    ivB64:         bytesToBase64(iv),
    plainHashB64:  bytesToBase64(new Uint8Array(plainHashBuffer)),
    cipherHashB64: bytesToBase64(new Uint8Array(cipherHashBuffer)),
    keyB64:        bytesToBase64(new Uint8Array(rawKey)),
  }
}
```

Listing 3: PBKDF2-SHA256 key derivation at 600,000 iterations (OWASP password-storage guidance; exceeds the NIST SP 800-132 minimum work factor). Derives the AES-256-GCM wrapping key protecting both ECIES and ECDSA private keys in `localStorage`.

```
// frontend/src/lib/crypto.ts
const PBKDF2_ITERATIONS = 600000
export async function derivePbkdf2Key(
    password: string, saltBase64: string
): Promise<CryptoKey> {
  const enc  = new TextEncoder()
  const salt = base64ToBytes(saltBase64)
  const base = await subtle.importKey(
      'raw', enc.encode(password),
      'PBKDF2', false, ['deriveKey'])
  return subtle.deriveKey(
    { name: 'PBKDF2', hash: 'SHA-256',
      salt: salt.buffer,
      iterations: PBKDF2_ITERATIONS },
    base,
    { name: 'AES-GCM', length: 256 },
    false, ['encrypt','decrypt'])
}
```

Listing 4: Server-side ECDSA P-256 verification. `SHA256withECDSAinP1363Format` natively consumes WebCrypto's raw $r\|s$ output without format conversion.

```
// EcdsaVerifier.java
public static boolean verify(
        String compositePayloadB64,
    String signatureB64,
    String publicKeyJwkJson) {
  try {
    byte[] compositePayload =
        Base64.getDecoder().decode(compositePayloadB64);
    byte[] sig =
        Base64.getDecoder().decode(signatureB64);
    PublicKey pub =
        parseJwkPublicKey(publicKeyJwkJson);
    Signature s = Signature.getInstance(
        "SHA256withECDSAinP1363Format");
    s.initVerify(pub);
    // P = H_D (32 B) || H_C (32 B) || UTF-8(id_D)
    // Prototype payload: content hashes + doc ID.
    // Owner, timestamp, filename not signed in
    // prototype; full metadata binding is future work.
    s.update(compositePayload);
    return s.verify(sig);
  } catch (Exception e) {
    log.warn("ECDSA verification error: {}",
        e.getMessage());
    return false;
  }
}
```

Listing 5: Strict fail-closed ACL in the document download path. Layer 1: Spring Security `@PreAuthorize`. Layer 2: `CheckAccess` chaincode. If Fabric is unreachable, the system strictly denies access and logs `FABRIC_OUTAGE_ACCESS_DENIED`.

```
// DocumentService.java - downloadCiphertext()
String requesterMspId = requester.getFirm() == null
    ? null
    : requester.getFirm().getMspId();
if (requesterMspId == null || requesterMspId.isBlank()) {
  throw new AccessDeniedException(
      "Requester has no firm/MSP binding.");
}
boolean allowed;
try {
  allowed = fabricGatewayService.checkAccess(
      docId.toString(),
      requester.getId().toString(),
      requesterMspId);
} catch (FabricException e) {
  auditService.log("FABRIC_OUTAGE_ACCESS_DENIED",
      requester.getId(),
      "DOCUMENT", docId.toString(), null,
      "{\"reason\":\"" + e.getMessage() + "\"}");
  throw new ServiceUnavailableException(
      "Fabric network unreachable. Access strictly denied.");
}
if (!allowed) throw new AccessDeniedException(
    "Access denied by chaincode.");
```

Listing 6: INSERT-only enforcement on `audit_log`. Row-level triggers block UPDATE and DELETE; a statement-level trigger blocks TRUNCATE (which bypasses row-level triggers in PostgreSQL). TRUNCATE privilege is also revoked from application roles.

```
-- 001-initial-schema.sql
CREATE OR REPLACE FUNCTION
    audit_log_prevent_modification()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  IF TG_OP = 'UPDATE' THEN
    RAISE EXCEPTION
      'audit_log is append-only: UPDATE not permitted';
  ELSIF TG_OP = 'DELETE' THEN
    RAISE EXCEPTION
      'audit_log is append-only: DELETE not permitted';
  END IF;
  RETURN NULL;
END; $$;

CREATE TRIGGER audit_log_no_modify
  BEFORE UPDATE OR DELETE ON audit_log
  FOR EACH ROW
  EXECUTE FUNCTION audit_log_prevent_modification();

-- TRUNCATE protection (statement-level;
-- row-level triggers cannot catch TRUNCATE)
```

```
CREATE OR REPLACE FUNCTION
    audit_log_prevent_truncate()
RETURNS TRIGGER LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION
    'audit_log is append-only: TRUNCATE not permitted';
  RETURN NULL;
END; $$;

CREATE TRIGGER audit_log_no_truncate
  BEFORE TRUNCATE ON audit_log
  FOR EACH STATEMENT
  EXECUTE FUNCTION audit_log_prevent_truncate();

-- Revoke TRUNCATE from application roles
REVOKE TRUNCATE ON audit_log FROM app_role;
```

CRediT authorship contribution statement

Golam Bari Fardeen: [roles]. **Namare Shakib Angkon:** [roles]. **Syed Zayan Anwar:** [roles]. **Md. Masum Mizi:** [roles]. **Shinthia Rahman:** [roles]. **Md. Motaharul Islam:** [roles].

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work was supported by the Institute for Advanced Research Publication Grant of United International University, under Grant IAR 2025 Pub 093.

Data availability

Data will be made available on request.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Claude (Anthropic) in order to assist with language editing, manuscript structuring and LaTeX formatting. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

[1] American Bar Association, 2023 cybersecurity tech report, accessed: Dec. 14, 2025 (Dec. 2023).
URL https://www.americanbar.org/groups/law_practice/resources/tech-report/2023/2023-cybersecurity-techreport/

[2] Verizon Business, 2025 data breach investigations report: Small- and Medium-Sized Business Snapshot, pDF. Accessed: Dec. 14, 2025 (May 2025).
URL https://www.verizon.com/business/resources/infographics/2025-dbir-smb-snapshot.pdf

[3] Information Commissioner's Office, Data security incident trends, [Online], accessed: Jun. 2026 (2024).
URL https://ico.org.uk/action-weve-taken/data-security-incident-trends/

[4] D. Yaga, P. Mell, N. Roby, K. Scarfone, Blockchain technology overview, NIST Internal Report 8202, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Oct. 2018). doi:10.6028/NIST.IR.8202.

[5] K. Kent, M. Souppaya, Guide to computer security log management, Special Publication 800-92, National Institute of Standards and Technology (Sep. 2006). doi:10.6028/NIST.SP.800-92.

[6] Joint Task Force, Security and privacy controls for information systems and organizations, NIST Special Publication 800-53 Revision 5 (Update 1), National Institute of Standards and Technology (NIST) (Sep. 2020). doi:10.6028/NIST.SP.800-53r5.

[7] D. Batista, A. L. Mangeth, I. Frajhof, P. H. Alves, R. Nasser, G. Robichez, G. M. Silva, F. P. d. Miranda, Exploring blockchain technology for chain of custody control in physical evidence: A systematic literature review, Journal of Risk and Financial Management 16 (8) (2023). doi:10.3390/jrfm16080360.

[8] Y. Cai, M. Feng, A time-bound data access control scheme based on attribute-based encryption, in: Proceedings of the 8th International Conference on Cyber Security and Information Engineering, ICCSIE '23, Association for Computing Machinery, 2023, pp. 52–56. doi:10.1145/3617184.3617781.

[9] A. Ahmad, M. Saad, A. Mohaisen, Secure and transparent audit logs with blockaudit, Journal of Network and Computer Applications 145 (2019) 102406. doi:10.1016/j.jnca.2019.102406.

[10] E. Androulaki, A. Barger, V. Bortnikov, C. Cachin, K. Christidis, A. De Caro, D. Enyeart, C. Ferris, G. Laventman, Y. Manevich, S. Muralidharan, C. Murthy, B. Nguyen, M. Sethi, G. Singh, K. Smith, A. Sorniotti, C. Stathakopoulou, M. Vukolić, S. W. Cocco, J. Yellick, Hyperledger fabric: A distributed operating system for permissioned blockchains, in: Proceedings of the Thirteenth EuroSys Conference (EuroSys '18), ACM, 2018, pp. 1–15. doi:10.1145/3190508.3190538.

[11] J. Benet, IPFS: Content addressed, versioned, P2P file system, arXiv preprint arXiv:1407.3561, accessed: Dec. 14, 2025 (2014).
URL https://arxiv.org/abs/1407.3561

[12] Hyperledger Fabric Documentation Contributors, The ordering service, hyperledger Fabric documentation (release-2.4). Licensed under CC BY 4.0. Revision 2a135189. Accessed: Dec. 25, 2025 (2022).
URL https://hyperledger-fabric.readthedocs.io/en/release-2.4/orderer/ordering_service.html

[13] D. Ongaro, J. Ousterhout, In search of an understandable consensus algorithm, in: 2014 USENIX Annual Technical Conference (USENIX ATC 14), USENIX Association, Philadelphia, PA, 2014, pp. 305–319.

[14] M. J. Dworkin, E. Barker, J. R. Nechvatal, J. Foti, L. E. Bassham, E. Roback, J. F. Dray Jr., Advanced encryption standard (AES), Federal

Information Processing Standards Publication (FIPS) 197 (Nov. 2001). doi:10.6028/NIST.FIPS.197.

[15] H. Halpin, The W3C web cryptography API: Motivation and overview, in: Proceedings of the 23rd International Conference on World Wide Web, WWW '14 Companion, Association for Computing Machinery, 2014, pp. 959–964. doi:10.1145/2567948.2579224.

[16] L. Chen, D. Moody, A. Regenscheid, A. Robinson, Digital signature standard (DSS), Federal Information Processing Standards Publication (FIPS) 186-5 (Feb. 2023). doi:10.6028/NIST.FIPS.186-5.

[17] J. Nielsen, Usability Engineering, Morgan Kaufmann, San Francisco, CA, USA, 1993.

[18] R. B. Miller, Response time in man-computer conversational transactions, in: Proceedings of the December 9-11, 1968, Fall Joint Computer Conference, Part I, AFIPS '68 (Fall, part I), Association for Computing Machinery, New York, NY, USA, 1968, p. 267–277. doi:10.1145/1476589.1476628.
URL https://doi.org/10.1145/1476589.1476628

[19] International Legal Technology Association, ILTA technology survey 2023, Tech. rep., ILTA (2023).
URL https://www.iltanet.org/research/surveys

[20] J. Abou Jaoude, R. G. Saade, Blockchain applications–usage in different domains, IEEE Access 7 (2019) 45360–45381. doi:10.1109/ACCESS.2019.2902501.

[21] F. Casino, T. K. Dasaklis, C. Patsakis, A systematic literature review of blockchain-based applications: Current status, classification and open issues, Telematics and Informatics 36 (2019) 55–81. doi:10.1016/j.tele.2018.11.006.

[22] H. Alanzi, M. Alkhatib, Towards improving privacy and security of identity management systems using blockchain technology: A systematic review, Applied Sciences 12 (2022) 12415. doi:10.3390/app122312415.

[23] H. Precht, J. Hüllmann, J. Marx Gómez, Paperless everything: A systematic literature review for the design of blockchain-based document

management systems, Distributed Ledger Technologies: Research and Practice 5 (2) (Jun. 2026). doi:10.1145/3737296.

[24] S. Liu, Q. Zheng, A study of a blockchain-based judicial evidence preservation scheme, Blockchain: Research and Applications 5 (2) (2024) 100192. doi:10.1016/j.bcra.2024.100192.

[25] D. Kim, S.-Y. Ihm, Y. Son, Two-level blockchain system for digital crime evidence management, Sensors 21 (9) (2021). doi:10.3390/s21093051.

[26] B. Onyeashie, M. Abubakar, P. Leimich, S. Mckeown, G. Russell, Privacy-preserving and scalable digital evidence management: A hyperledger fabric architecture with growth projections for law enforcement, in: 2025 International Conference on New Trends in Computing Sciences (ICTCS), 2025, pp. 105–112. doi:10.1109/ICTCS65341.2025.10989348.

[27] A. Verma, P. Bhattacharya, D. Saraswat, S. Tanwar, NyaYa: Blockchain-based electronic law record management scheme for judicial investigations, Journal of Information Security and Applications 63 (2021) 103025. doi:10.1016/j.jisa.2021.103025.

[28] J. Hanafi, Y. Prayudi, A. Luthfi, IPFSChain: Interplanetary file system and hyperledger fabric collaboration for chain of custody and digital evidence management, International Journal of Computer Applications 183 (2021) 24–31. doi:10.5120/ijca2021921808.

[29] J. Guo, K. Zhao, Z. Liang, K. Min, Efficient and secure emr storage and sharing scheme based on hyperledger fabric and ipfs, Applied Sciences 14 (2024) 5005. doi:10.3390/app14125005.

[30] A. Birgisson, J. G. Politz, Ú. Erlingsson, A. Taly, M. Vrable, M. Lentczner, Macaroons: Cookies with contextual caveats for decentralized authorization in the cloud, in: Network and Distributed System Security Symposium, 2014.

[31] R. Pang, R. Caceres, M. Burrows, Z. Chen, P. Dave, N. Germer, A. Golynski, K. Graney, N. Kang, L. Kissner, J. L. Korn, A. Parmar, C. D. Richards, M. Wang, Zanzibar: Google's consistent, global authorization system, in: 2019 USENIX Annual Technical Conference (USENIX ATC '19), USENIX Association, Renton, WA, 2019.

[32] X. Qian, W. Wu, An efficient ciphertext policy attribute-based encryption scheme from lattices and its implementation, in: 2021 IEEE 6th International Conference on Computer and Communication Systems (ICCCS), 2021, pp. 732–742. doi:10.1109/ICCCS52626.2021.9449182.

[33] Hyperledger Fabric Documentation Contributors, Membership service provider (MSP), hyperledger Fabric documentation (latest). Licensed under CC BY 4.0. (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/membership/membership.html

[34] Hyperledger Fabric Documentation Contributors, Private data, hyperledger Fabric documentation (latest). Licensed under CC BY 4.0. (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/private-data/private-data.html

[35] Hyperledger Fabric Documentation Contributors, Endorsement policies, hyperledger Fabric documentation (release-2.4). Licensed under CC BY 4.0. (2022).
URL https://hyperledger-fabric.readthedocs.io/en/release-2.4/endorsement-policies.html

[36] IPFS Docs, Pin a file with IPFS, quickstart guide (Last updated: 2025-12-15) (Dec. 2025).
URL https://docs.ipfs.tech/quickstart/pin/

[37] IPFS Docs, Using garbage collection in Kubo, iPFS documentation page (Kubo tutorials) (Dec. 2025).
URL https://docs.ipfs.tech/how-to/kubo-garbage-collection/

[38] A. Lone, Forensic-chain: Ethereum blockchain based digital forensics chain of custody, Scientific and practical cyber security journal 1 (12 2017).

[39] M. Cebe, E. Erdin, K. Akkaya, H. Aksu, S. Uluagac, Block4forensic: An integrated lightweight blockchain framework for forensics applications of connected vehicles, IEEE Communications Magazine 56 (10) (2018) 50–57. doi:10.1109/MCOM.2018.1800137.

[40] M. Jlil, K. Jouti, C. Loqman, Blockchain and smart contracts based system for criminal record management, Indonesian Journal of Electrical

Engineering and Computer Science 37 (1) (2025) 365–379. doi:10.11591/ijeecs.v37.i1.pp365-379.

[41] V. Hu, D. Ferraiolo, D. Kuhn, A. Schnitzer, K. Sandlin, R. Miller, K. Scarfone, Guide to attribute based access control (abac) definition and considerations, National Institute of Standards and Technology Special Publication (2014) 162–800.

[42] U. Waheed, S. A. Khan, M. Masud, H. Jamshed, T. A. Jumani, N. U. R. Malik, Blockchain-based, dynamic attribute-based access control for smart home energy systems, Energies 18 (8) (2025). doi:10.3390/en18081973.

[43] Hyperledger Fabric Documentation Contributors, What's new in hyperledger fabric, accessed: Dec. 26, 2025 (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/whatsnew.html

[44] Hyperledger Fabric Documentation Contributors, The ordering service, hyperledger Fabric documentation. Accessed: Jun. 20, 2026 (2026).
URL https://hyperledger-fabric.readthedocs.io/en/latest/orderer/ordering_service.html

[45] Hyperledger Fabric CA Documentation Contributors, Welcome to hyperledger fabric CA (certificate authority), accessed: Dec. 26, 2025 (2024).
URL https://hyperledger-fabric-ca.readthedocs.io/en/latest/

[46] E. Barker, A. Roginsky, Transitioning the use of cryptographic algorithms and key lengths, NIST Special Publication 800-131A Revision 2, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Mar. 2019). doi:10.6028/NIST.SP.800-131Ar2.

[47] R. Sandhu, E. Coyne, H. Feinstein, C. Youman, Role-based access control models, Computer 29 (2) (1996) 38–47. doi:10.1109/2.485845.

[48] M. Dworkin, Recommendation for block cipher modes of operation: Galois/counter mode (GCM) and GMAC, NIST Special Publication 800-38D, National Institute of Standards and Technology (NIST) (Nov. 2007). doi:10.6028/NIST.SP.800-38D.

[49] V. Shoup, A proposal for an iso standard for public key encryption, Cryptology ePrint Archive 2001/112, International Association for Cryptologic Research (2001).
URL https://eprint.iacr.org/2001/112

[50] IEEE Standards Association, IEEE standard specifications for public-key cryptography – amendment 1: Additional techniques, IEEE Standard 1363a-2004, Institute of Electrical and Electronics Engineers, defines ECIES (Elliptic Curve Integrated Encryption Scheme). Accessed: Jan. 2026 (2004). doi:10.1109/IEEESTD.2004.94612.

[51] A. Rundgren, B. Jordan, S. Erdtman, JSON canonicalization scheme (JCS), RFC 8785, IETF (2020).
URL https://www.rfc-editor.org/rfc/rfc8785

[52] OWASP Foundation, Password storage cheat sheet, https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (2024).

[53] M. Sönmez Turan, E. Barker, W. Burr, L. Chen, Recommendation for password-based key derivation: Part 1: Storage applications, NIST Special Publication 800-132, National Institute of Standards and Technology (NIST), Gaithersburg, MD (Dec. 2010). doi:10.6028/NIST.SP.800-132.

[54] B. Kaliski, J. Jonsson, A. Rusch, PKCS #1: RSA cryptography specifications version 2.2, Request for Comments 8017, RFC Editor (Nov. 2016). doi:10.17487/RFC8017.

[55] K. O.-B. O. Agyekum, Q. Xia, E. B. Sifah, C. N. A. Cobblah, H. Xia, J. Gao, A proxy re-encryption approach to secure data sharing in the internet of things based on blockchain, IEEE Systems Journal 16 (1) (2022) 1685–1696. doi:10.1109/JSYST.2021.3076759.

[56] Y. Kim, S.-O. Lee, K. Ko, Secure outsourced blockchain-based medical data sharing system using proxy re-encryption, Applied Sciences 11 (2021) 9422. doi:10.3390/app11209422.

[57] M. Cooper, K. B. Schaffer, Security requirements for cryptographic modules, Federal Information Processing Standards Publication (FIPS) 140-3 (Mar. 2019). doi:10.6028/NIST.FIPS.140-3.

[58] D. Temoshok, J. Fenton, N. Lefkovitz, A. Regenscheid, J. Richer, P. Grassi, Digital identity guidelines, NIST Special Publication 800-63-4, National Institute of Standards and Technology (NIST), supersedes SP 800-63-3 (withdrawn 2025-08-01). Accessed: Jan. 2026 (Jul. 2025). doi:10.6028/NIST.SP.800-63-4.

[59] A. Kumar, J. Hodges, J. Jones, M. B. Jones, G. Mandyam, Web authentication: An api for accessing public key credentials –Level 3, Candidate recommendation snapshot, W3C (2026).
URL https://www.w3.org/TR/webauthn-3/

[60] M. B. Jones, J. Bradley, N. Sakimura, JSON Web Token (JWT), RFC 7519 (May 2015). doi:10.17487/RFC7519.

[61] Hyperledger Fabric Documentation Contributors, Configuring and operating a BFT ordering service, hyperledger Fabric documentation. Accessed: Jun. 20, 2026 (2026).
URL https://hyperledger-fabric.readthedocs.io/en/latest/bft_configuration.html

[62] P. Grassi, M. Garcia, J. Fenton, Digital identity guidelines, NIST Special Publication 800-63-3, National Institute of Standards and Technology (NIST), includes updates as of 2020-03-02. Withdrawn 2025-08-01; superseded by SP 800-63-4. Accessed: Dec. 20, 2025 (Jun. 2017). doi:10.6028/NIST.SP.800-63-3.

[63] Hyperledger Fabric Documentation Contributors, Fabric chaincode lifecycle, accessed: Dec. 26, 2025 (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/chaincode_lifecycle.html

[64] E. Politou, F. Casino, E. Alepis, C. Patsakis, Blockchain mutability: Challenges and proposed solutions, IEEE Transactions on Emerging Topics in Computing 9 (4) (2021) 1972–1986. doi:10.1109/TETC.2019.2949510.

[65] M. Finck, Blockchain and the general data protection regulation: Can distributed ledgers be squared with european data protection law?, Study PE 634.445, European Parliamentary Research Service (EPRS), European Parliament (Jul. 2019). doi:10.2861/535.

[66] Hyperledger Fabric Documentation Contributors, CouchDB as the state database, accessed: Dec. 26, 2025 (2022).
URL https://hyperledger-fabric.readthedocs.io/en/release-2.4/couchdb_as_state_database.html

[67] P. Webb, D. Syer, J. Long, S. Nicoll, R. Winch, A. Wilkinson, M. Halbritter, Spring boot reference documentation, version 3.2.x, vMware, Inc. Accessed: Jan. 2026 (2024).
URL https://docs.spring.io/spring-boot/docs/3.2.12/reference/html/

[68] The PostgreSQL Global Development Group, PostgreSQL 16 documentation: Trigger functions, accessed: Jan. 2026 (2024).
URL https://www.postgresql.org/docs/16/plpgsql-trigger.html

[69] Hyperledger Fabric Documentation Contributors, Wallet, accessed: Dec. 26, 2025 (2025).
URL https://hyperledger.github.io/fabric-sdk-node/release-2.2/module-fabric-network.Wallets.html

[70] O. Abdullah Lajam, T. Ahmed Helmy, Performance evaluation of IPFS in private networks, in: Proceedings of the 2021 4th International Conference on Data Storage and Data Engineering, DSDE '21, Association for Computing Machinery, 2021, pp. 77–84. doi:10.1145/3456146.3456159.

[71] Hyperledger Fabric Contributors, fabric-shim.ChaincodeStub class (hyperledger fabric contract api), aPI documentation (fabric-chaincode-node) (2025).
URL https://hyperledger.github.io/fabric-chaincode-node/main/api/fabric-shim.ChaincodeStub.html

[72] LF Decentralized Trust, Hyperledger caliper, project page (blockchain performance benchmarking tool). Accessed: Dec. 12, 2025 (2021).
URL https://www.hyperledger.org/use/caliper

[73] A. Woznica, M. Kędziora, Performance and scalability evaluation of a permissioned blockchain based on the hyperledger fabric, sawtooth and

iroha, Computer Science and Information Systems 19 (2022) 659–678. doi:10.2298/csis210507002w.

[74] A. Thakur, V. Ranga, R. Agarwal, Workload dynamics implications in permissioned blockchain scalability and performance, Cluster Computing 27 (2024) 11569–11593. doi:10.1007/s10586-024-04550-z.

[75] T. Benson, A. Akella, D. A. Maltz, Network traffic characteristics of data centers in the wild, in: Proceedings of the 10th ACM SIGCOMM Conference on Internet Measurement, IMC '10, Association for Computing Machinery, New York, NY, USA, 2010, p. 267–280. doi:10.1145/1879141.1879175.
URL https://doi.org/10.1145/1879141.1879175

[76] P. Thakkar, S. Nathan, B. Viswanathan, Performance benchmarking and optimizing hyperledger fabric blockchain platform, in: 2018 IEEE 26th International Symposium on Modeling, Analysis, and Simulation of Computer and Telecommunication Systems (MASCOTS), IEEE, 2018, pp. 264–276. doi:10.1109/MASCOTS.2018.00034.

[77] G. Ateniese, B. Magri, D. Venturi, E. Andrade, Redactable blockchain – or – rewriting history in Bitcoin and friends, in: 2017 IEEE European Symposium on Security and Privacy (EuroS&P), IEEE, 2017, pp. 111–126. doi:10.1109/EuroSP.2017.23.

[78] E. Barker, Recommendation for key management: Part 1 – general, NIST Special Publication 800-57 Part 1 Revision 5, National Institute of Standards and Technology (NIST), Gaithersburg, MD (May 2020). doi:10.6028/NIST.SP.800-57pt1r5.

[79] Hyperledger Fabric Documentation Contributors, Channels, hyperledger Fabric documentation (latest). Licensed under CC BY 4.0. (2025).
URL https://hyperledger-fabric.readthedocs.io/en/latest/channels.html

[80] J. Groth, On the size of pairing-based non-interactive arguments, in: Proceedings of the 35th Annual International Conference on Advances in Cryptology–EUROCRYPT 2016, Part II, Springer-Verlag, Berlin, Heidelberg, 2016, pp. 305–326.

[81] European Parliament and Council of the European Union, Regulation (EU) 2016/679 of the European Parliament and of the Council on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation), Regulation 2016/679, Official Journal of the European Union, oJ L 119, 4 May 2016, pp. 1–88. In force since 25 May 2018. Accessed: Jan. 2026 (Apr. 2016).
URL https://eur-lex.europa.eu/legal-content/EN/TXT/?uri=CELEX:32016R0679

[82] T. Lyons, L. Courcelas, K. Timsit, Blockchain and the GDPR, Thematic report, European Union Blockchain Observatory and Forum (Oct. 2018).
URL https://blockchain-observatory.ec.europa.eu/document/download/2df0bb3c-61fd-4ac6-921b-ff50fc32aeb9_en

[83] European Data Protection Board, Guidelines 01/2025 on pseudonymisation, Guidelines 01/2025, European Data Protection Board, guidelines 01/2025. Draft for public consultation adopted 16 January 2025; consultation closed 14 March 2025; finalisation pending at time of submission (January 2025).
URL https://www.edpb.europa.eu/system/files/2025-01/edpb_guidelines_202501_pseudonymisation_en.pdf

[84] M. Finck, F. Pallas, They who must not be identified–distinguishing personal from non-personal data under the gdpr, International Data Privacy Law 10 (1) (2020) 11–36. doi:10.1093/idpl/ipz026.

[85] A. Permenev, D. Dimitrov, P. Tsankov, D. Drachsler-Cohen, M. Vechev, VerX: Safety verification of smart contracts, in: 2020 IEEE Symposium on Security and Privacy (SP), IEEE, 2020, pp. 1661–1677. doi:10.1109/SP40000.2020.00024.

[86] S. Siddamsetti, Anomaly detection in blockchain using machine learning, Journal of Electrical Systems 20 (3) (2024) 619–634. doi:10.52783/jes.2988.

[87] M. Sporny, A. Guy, M. Sabadello, D. Reed, Decentralized identifiers (DIDs) v1.0: Core architecture, data model, and representations, W3C Recommendation, accessed: Dec. 14, 2025 (Jul. 2022).
URL https://www.w3.org/TR/did-1.0/

[88] M. Sporny, D. Longley, D. Chadwick, Verifiable credentials data model v1.1, W3C Recommendation, accessed: Jan. 2026 (Mar. 2022).
URL https://www.w3.org/TR/vc-data-model/